

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

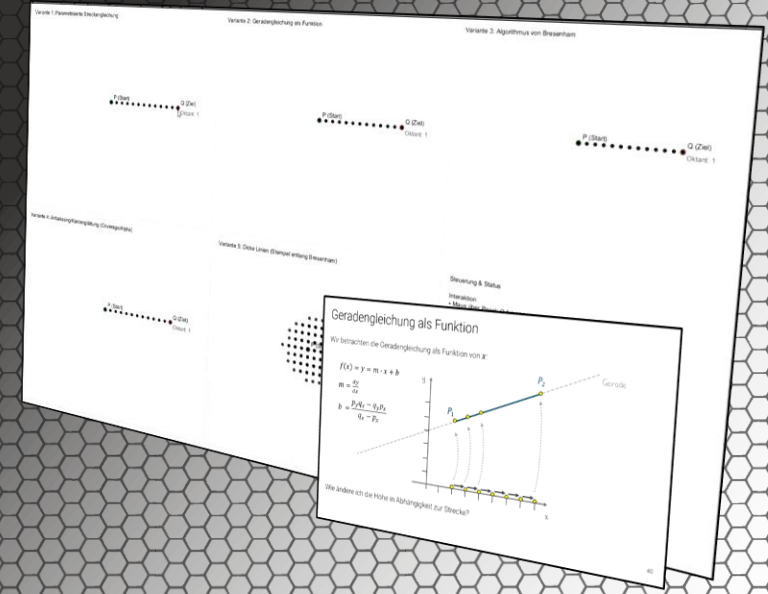
Sommersemester 2026



Linien unter der Lupe

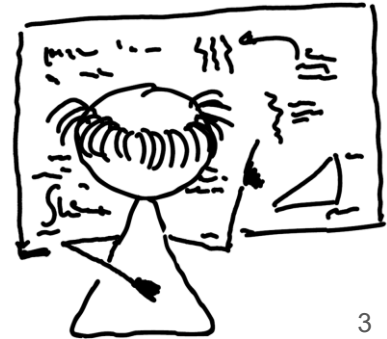
„Eine Linie ist ein Punkt der spazieren geht.“ (Paul Klee)

PD Prof. Dr. Marco Block-Berlitz



Ziele und Inhalt

- Parametrisierte Geradengleichung
- Umformung zur parametrisierten Streckengleichung
- Schrittweite der Punkte ermitteln
- Variante 1: Parametrisierte Streckengleichung
- Betrachtung der Geradengleichung als Funktion
- Steigung und Schnittpunkt berechnen
- Variante 2: Geradengleichung als Funktion
- Problematik: Große Steigungen
- Lösung durch Vertauschen der Achsen
- Oktanten und Steigungen
- Variante 3: Algorithmus von Bresenham
- Steigung und Fehler
- Implementierungen der Varianten 4 und 5
- Vergleich der drei Varianten
- Aliasing-Effekt
- Interpretation des Fehlers
- Überlappungskoeffizienten ableiten
- Antialiasing/Kantenglättung
- Breite Linien über Pixel-Duplikation
- Breite Linien über Kreis-Variante
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



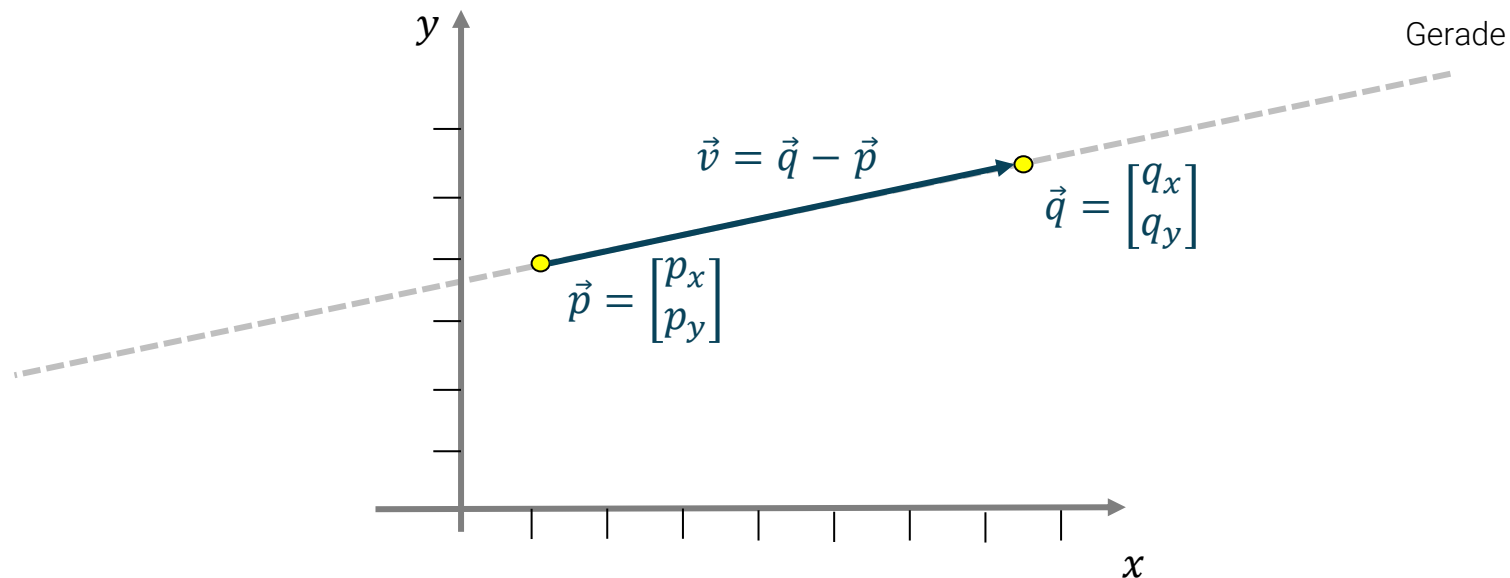


Wir zeichnen eine Linie mit der
parametrisierten Geradengleichung

Parametrisierte Geradengleichung

Wir können alle Punkte auf einer Geraden durch die [parametrisierte Geradengleichung](#) erzeugen:

$$g: \vec{u} = \vec{p} + r \cdot \vec{v}, \text{ mit } r \in \mathbb{R}$$



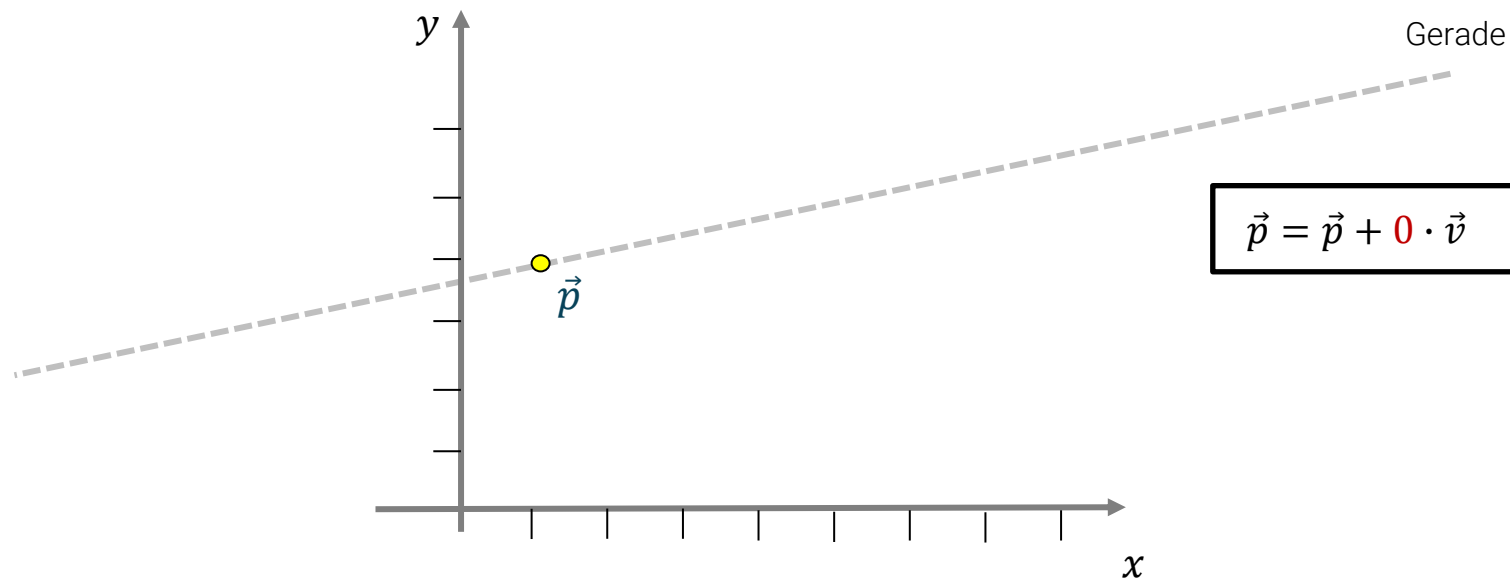


Können wir alle Punkte auf
der Geraden erzeugen?

Parametrisierte Geradengleichung

Wir können alle Punkte auf einer Geraden durch die [parametrisierte Geradengleichung](#) erzeugen:

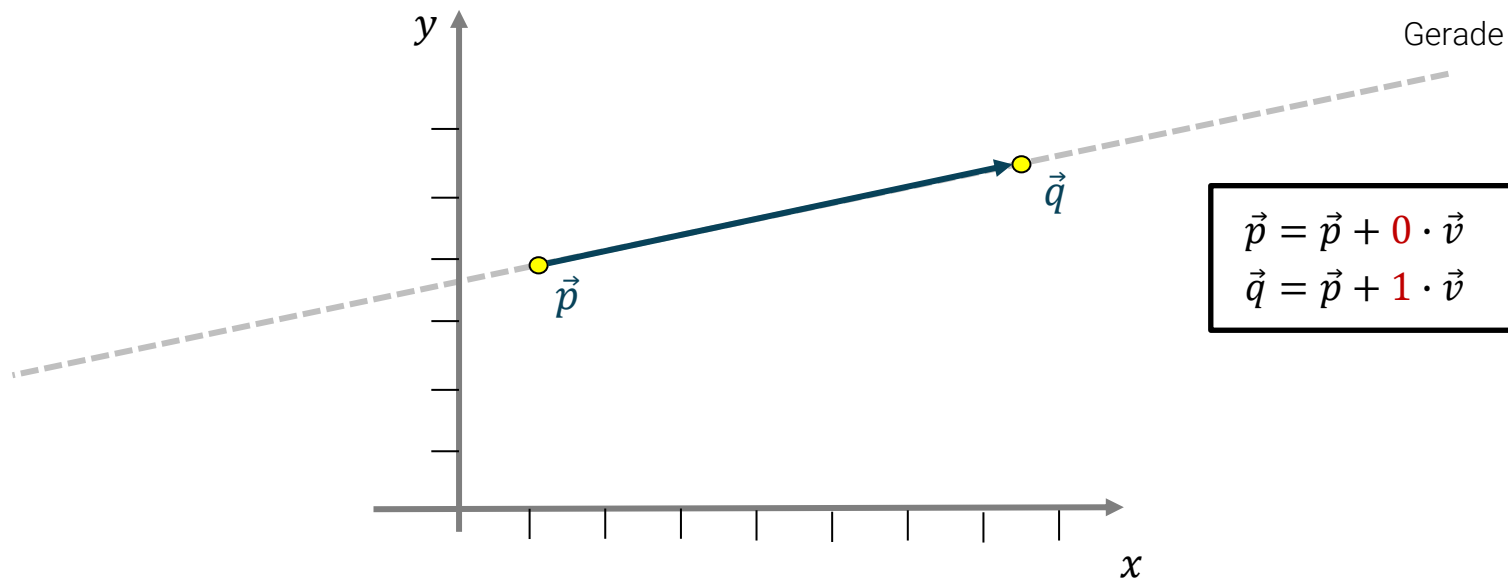
$g: \vec{u} = \vec{p} + r \cdot \vec{v}, \text{ mit } r \in \mathbb{R}$



Parametrisierte Geradengleichung

Wir können alle Punkte auf einer Geraden durch die [parametrisierte Geradengleichung](#) erzeugen:

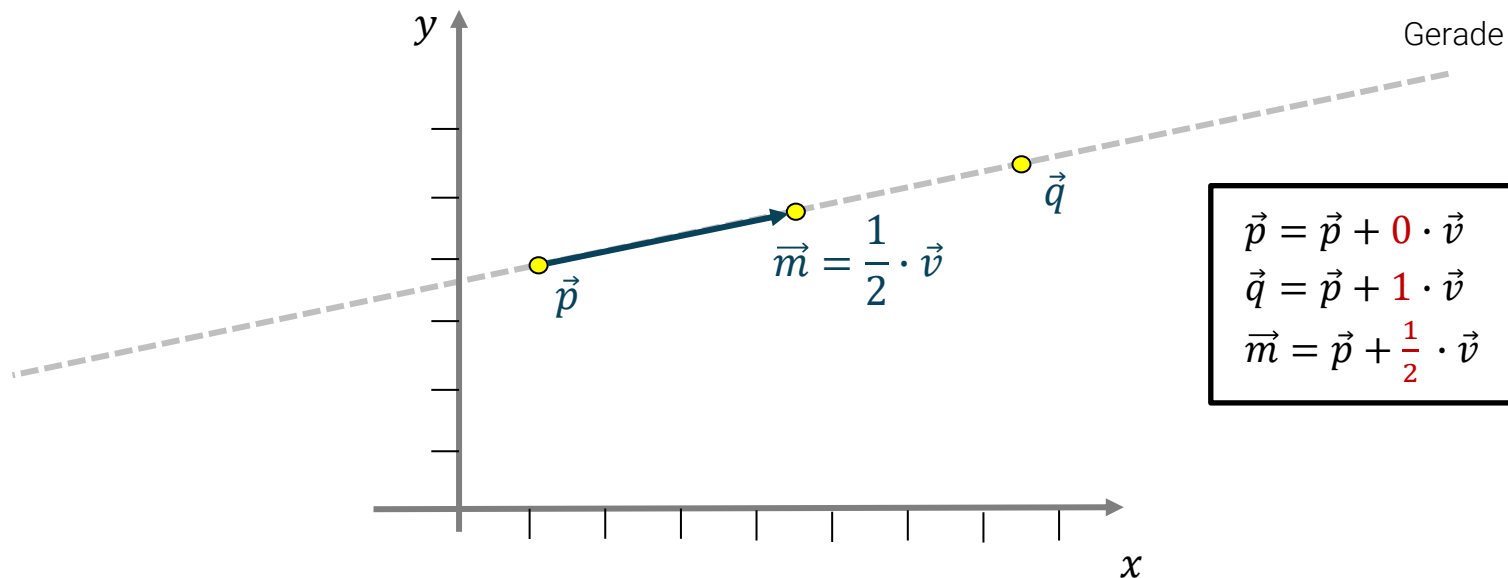
$g: \vec{u} = \vec{p} + r \cdot \vec{v}$, mit $r \in \mathbb{R}$



Parametrisierte Geradengleichung

Wir können alle Punkte auf einer Geraden durch die [parametrisierte Geradengleichung](#) erzeugen:

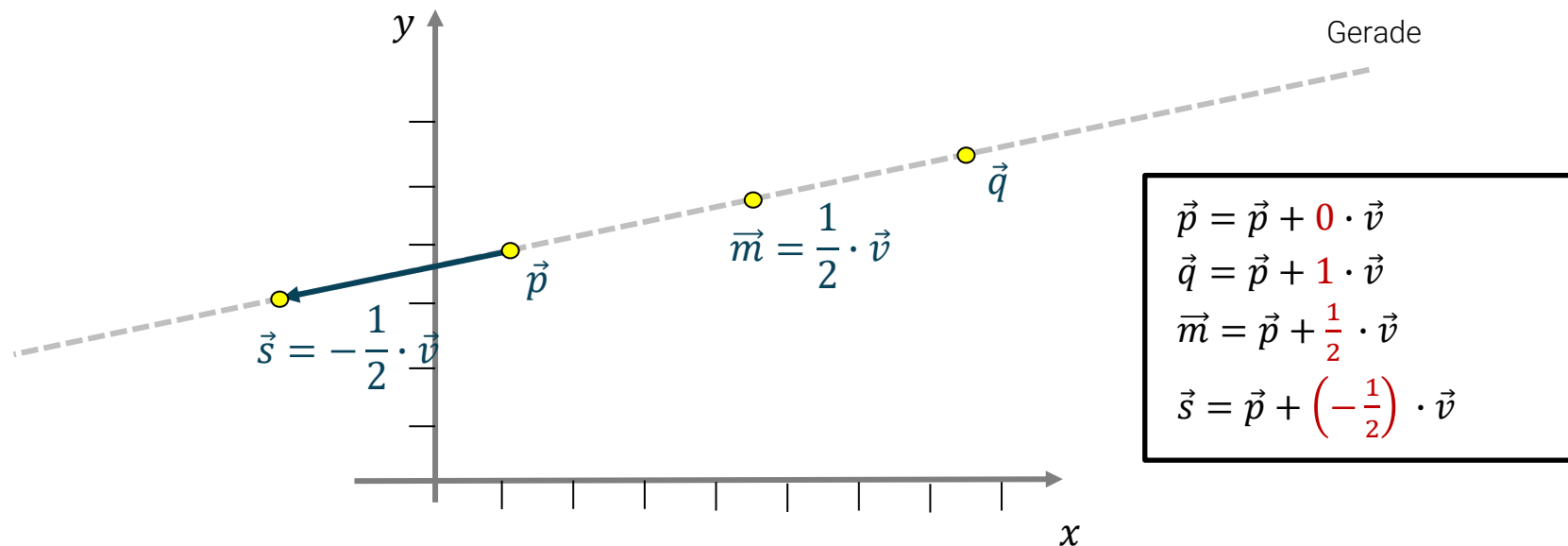
$g: \vec{u} = \vec{p} + r \cdot \vec{v}$, mit $r \in \mathbb{R}$



Parametrisierte Geradengleichung

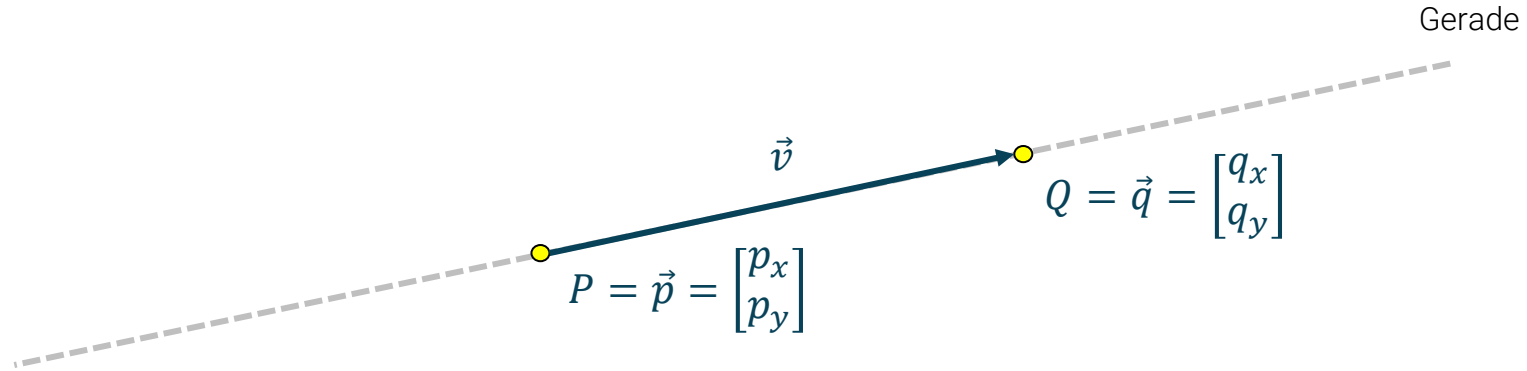
Wir können alle Punkte auf einer Geraden durch die [parametrisierte Geradengleichung](#) erzeugen:

$g: \vec{u} = \vec{p} + r \cdot \vec{v}$, mit $r \in \mathbb{R}$





Wie müssen wir die parametrisierte Geradengleichung anpassen, wenn wir nur die Linie **zwischen zwei Punkten** erzeugen wollen?



Wir konstruieren also über r die Vielfachen von $\vec{v}$:

$$g: \vec{u} = \vec{p} + r \cdot \vec{v}, \text{ mit } r \in \mathbb{R}$$

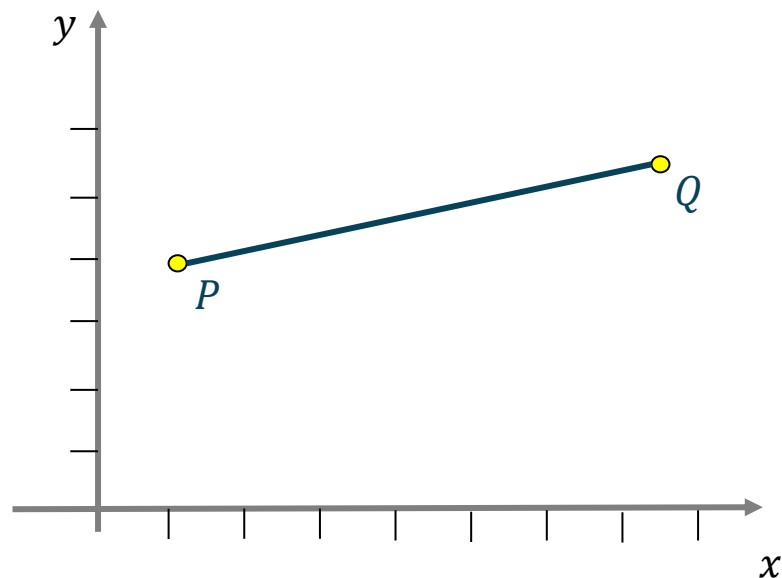
Wir interessieren uns aber **nur für die Strecke $\overline{PQ}$** . Wie müssen wir die parametrisierte Geradengleichung anpassen, um daraus eine parametrisierte Streckengleichung zu erhalten?

$$g: \vec{u} = \vec{p} + r \cdot \vec{v}, \text{ mit } r \in [0, 1]$$

Parametrisierte Streckengleichung (Lineare Interpolation)

Wir können alle Punkte auf einer Strecke $\overline{PQ}$ durch die **parametrisierte Streckengleichung** erzeugen:

$g: \vec{u} = \vec{p} + r \cdot \vec{v}$, mit $r \in [0,1]$ alternativ geht auch $L = (1 - t) \cdot P + t \cdot Q$, mit $t \in [0,1]$



Strecke

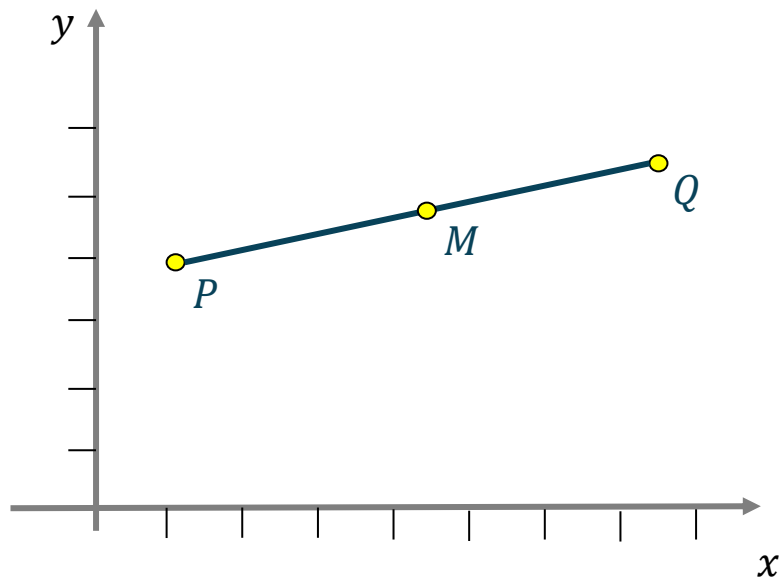
$$P = (1 - 0) \cdot P + 0 \cdot Q$$

$$Q = (1 - 1) \cdot P + 1 \cdot Q$$

Parametrisierte Streckengleichung

Wir können alle Punkte auf einer Strecke $\overline{PQ}$ durch die **parametrisierte Streckengleichung** erzeugen:

$g: \vec{u} = \vec{p} + r \cdot \vec{v}$, mit $r \in [0,1]$ alternativ geht auch $L = (1 - t) \cdot P + t \cdot Q$, mit $t \in [0,1]$



Strecke

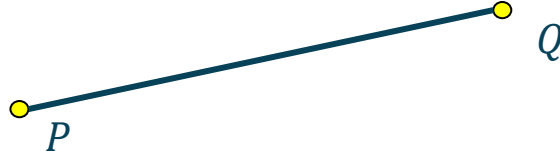
$$\begin{aligned} P &= (1 - 0) \cdot P + 0 \cdot Q \\ Q &= (1 - 1) \cdot P + 1 \cdot Q \\ M &= \left(1 - \frac{1}{2}\right) \cdot P + \frac{1}{2} \cdot Q \end{aligned}$$



Wie müssen wir t im Laufe der Zeit verändern?

$$L = (1 - t) \cdot P + t \cdot Q, \text{ mit } t \in [0,1]$$

Strecke



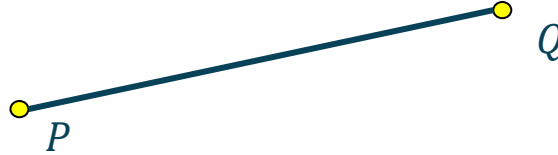
Wie müssen wir t im Laufe der Zeit verändern?

Wie viele Punkte liegen (in einer Rastergrafik) auf der Linie zwischen P und Q ?

$\|\overline{PQ}\|$? $\|\vec{v}\|$?

$$L = (1 - t) \cdot P + t \cdot Q, \text{ mit } t \in [0,1]$$

Strecke



Wie müssen wir t im Laufe der Zeit verändern?

Wie viele Punkte liegen (in einer Rastergrafik) auf der Linie zwischen P und Q ?

$$\|\overline{PQ}\| = \|\vec{v}\| = d = \sqrt{v_x^2 + v_y^2}$$

Wie bestimmen wir jetzt die Schrittweite für t über d ?

$$t = \frac{1}{d} = \frac{1}{\sqrt{v_x^2 + v_y^2}}$$

Variante 1: Linienzeichnen über die parametrisierte Streckengleichung

Methode setPixel überladen

Gehen wir hier davon aus, dass **zwei Funktionen zum Pixelzeichnen** angeboten werden:

```
void setPixel(int x, int y) {  
    // schaltet das Pixel (x, y) an  
}  
  
void setPixel(double x, double y) {  
    setPixel((int) (x+0.5), (int) (y+0.5));  
}
```

Java

Damit erleichtert sich der Aufruf zum Anschalten eines Pixels für beide Varianten erheblich:

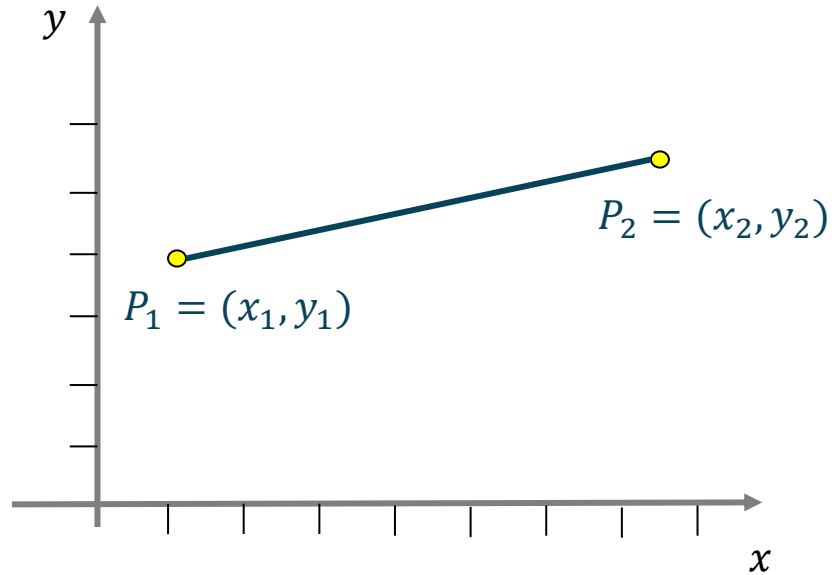
```
setPixel(3, 4);  
setPixel(6.3, 2.25);
```

Java



Linie zeichnen - Version 1

Schauen wir uns die Implementierung der ersten Variante des Linienzeichnens über die parametrisierte Streckengleichung an.

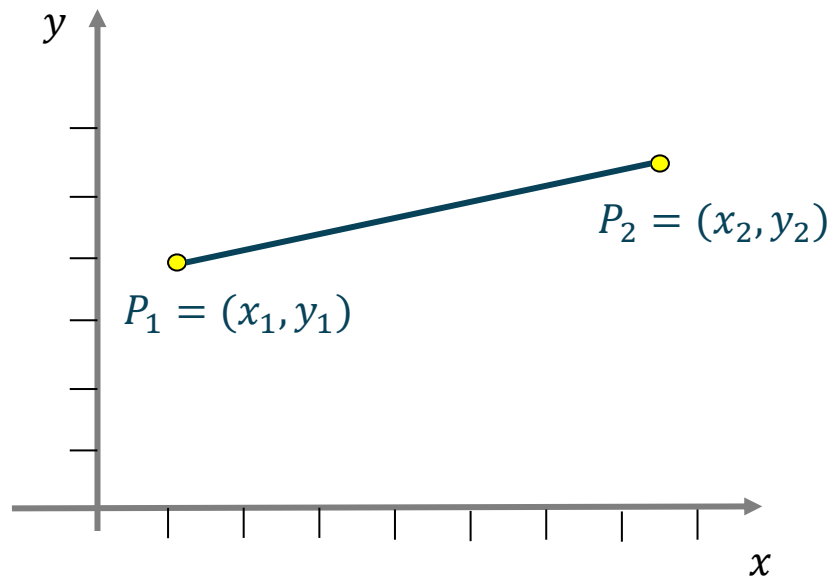


```
drawLineVersion1(x1, y1, x2, y2);
```

Java

Linie zeichnen - Version 1

Schauen wir uns die Implementierung der ersten Variante des Linienzeichnens über die parametrisierte Streckengleichung an.



```
void drawLineVersion1(int x1, int y1, int x2, int y2) {  
    int dx = x2 - x1;           // Weg in x  
    int dy = y2 - y1;           // Weg in y  
  
    // Schrittweite berechnen  
    double step = 1.0/Math.sqrt(dx*dx + dy*dy);  
  
    for (double t=0.0; t<=1.0; t+=step)  
        setPixel(x1+t*dx, y1+t*dy);    // setze Pixel  
}
```

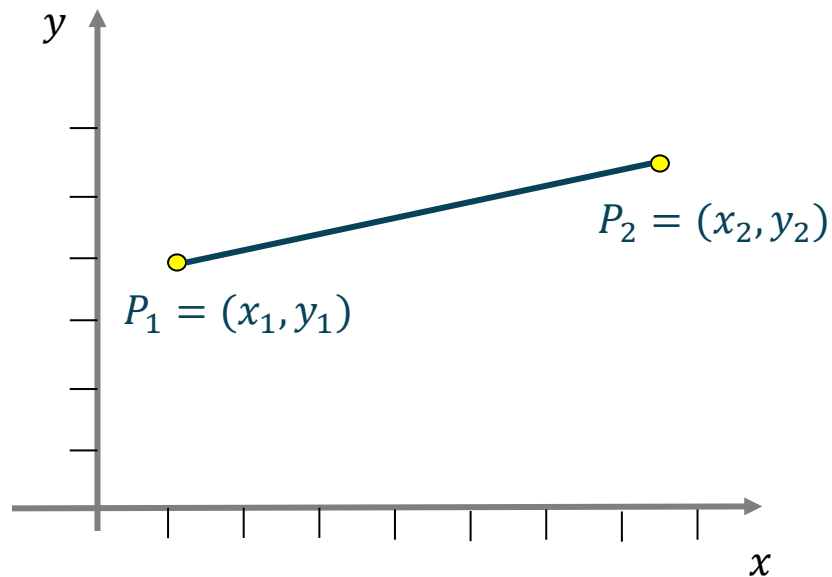
Java



Haben wir alle Pixel erwischt?

Linie zeichnen - Version 1

Schauen wir uns die Implementierung der ersten Variante des Linienzeichnens über die parametrisierte Streckengleichung an.



```
void drawLineVersion1(int x1, int y1, int x2, int y2) {  
    int dx = x2 - x1;           // Weg in x  
    int dy = y2 - y1;           // Weg in y  
  
    // Schrittweite berechnen  
    double step = 1.0/Math.sqrt(dx*dx + dy*dy);  
  
    for (double t=0.0; t<1.0; t+=step)  
        setPixel(x1+t*dx, y1+t*dy);    // setze Pixel  
  
    setPixel(x2, y2);  
}
```

Das Programm hat den Nachteil, dass wir **zu viel Gleitkommaarithmetik** verwenden. Wir verwenden diese Gleitkommaarithmetik sowohl für die Berechnung von x als auch für y .

Eine neue Idee:

Wir inkrementieren x schrittweise und
lassen uns y also $f(x)$ berechnen.

$$f(x) = m \cdot x + b$$

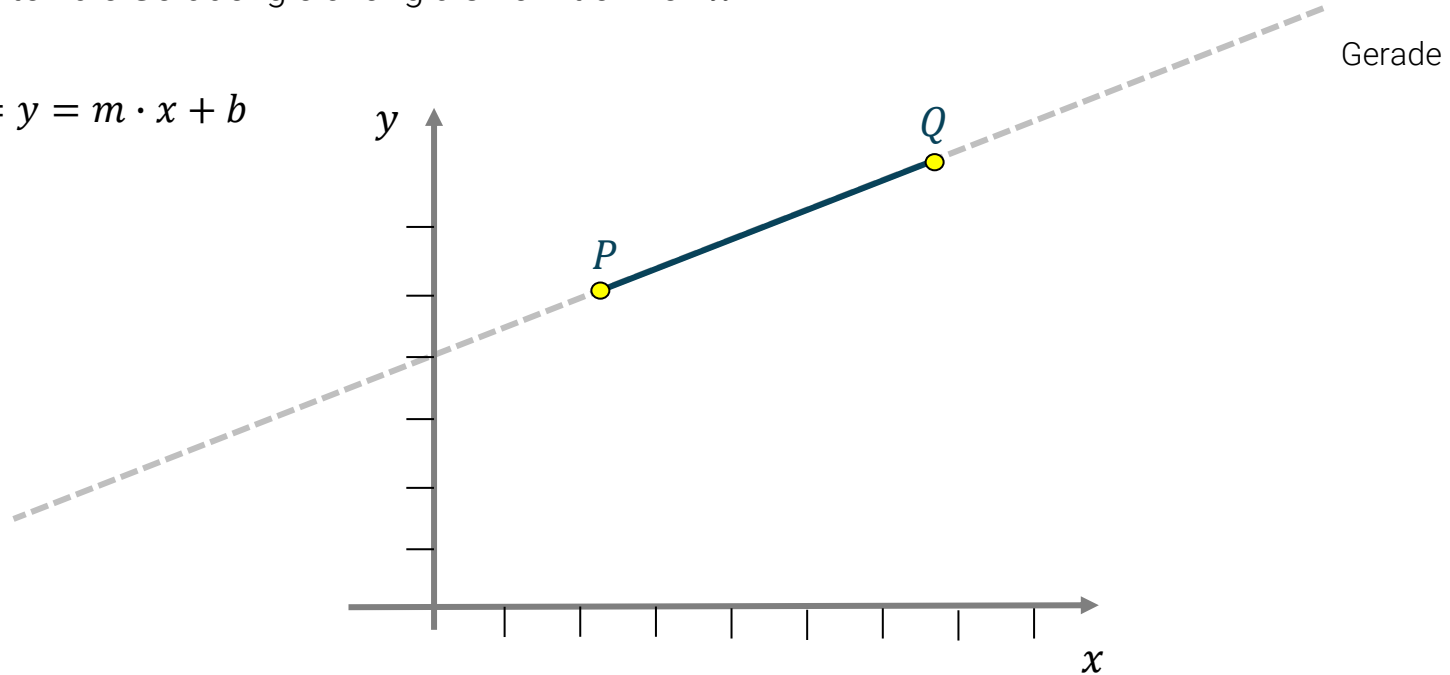
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = ?$$

$$b = ?$$



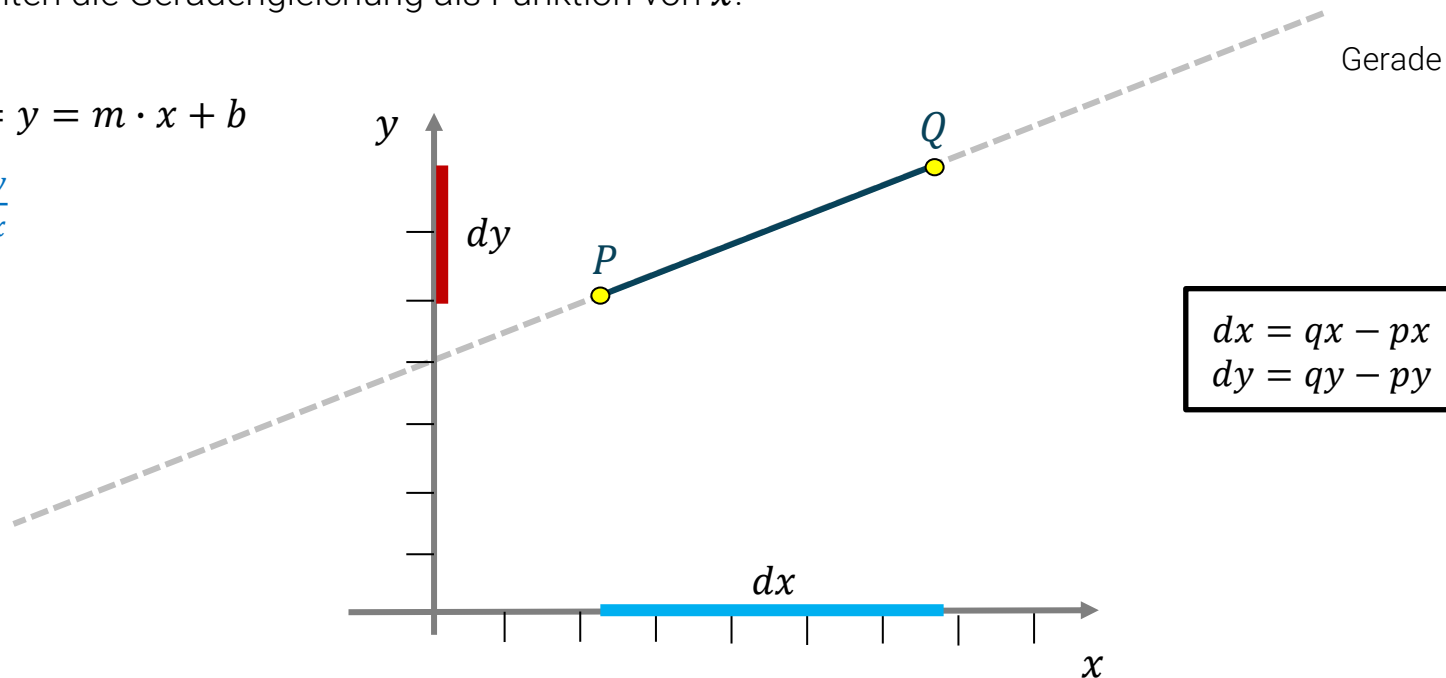
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = ?$$



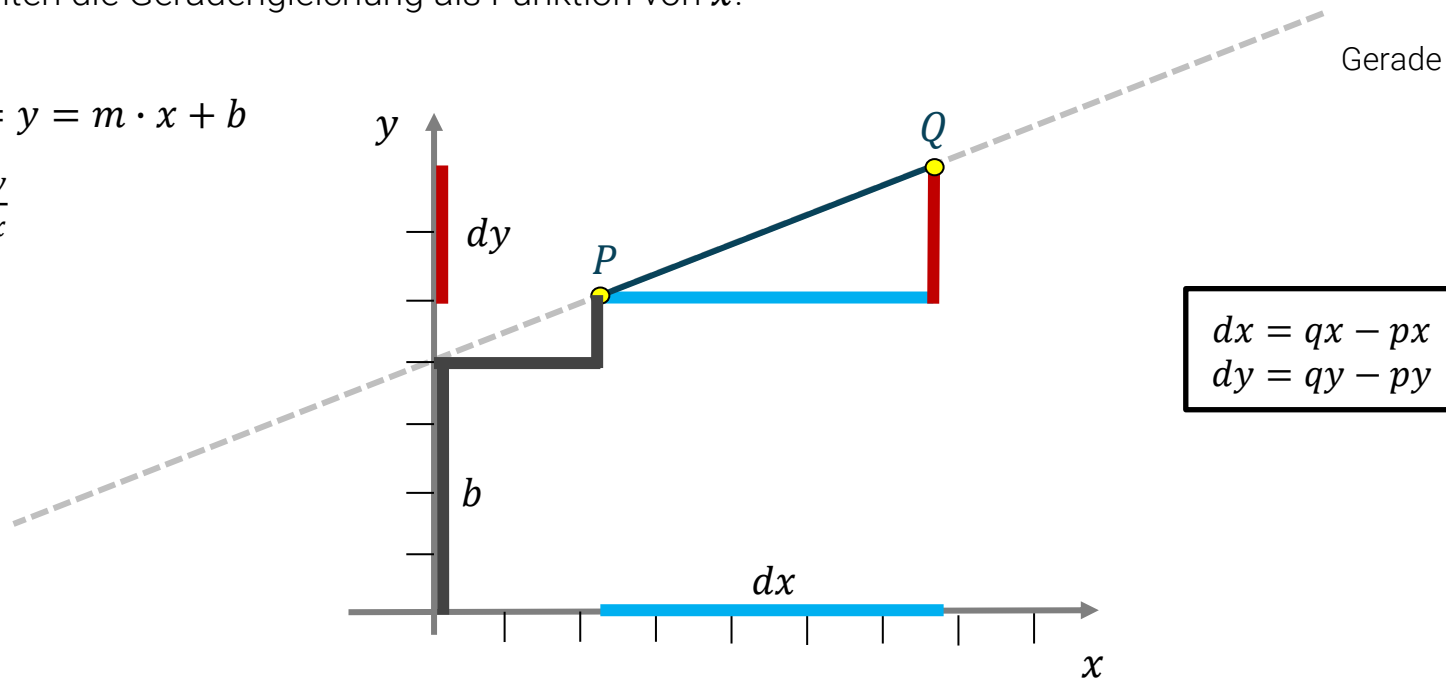
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = ?$$



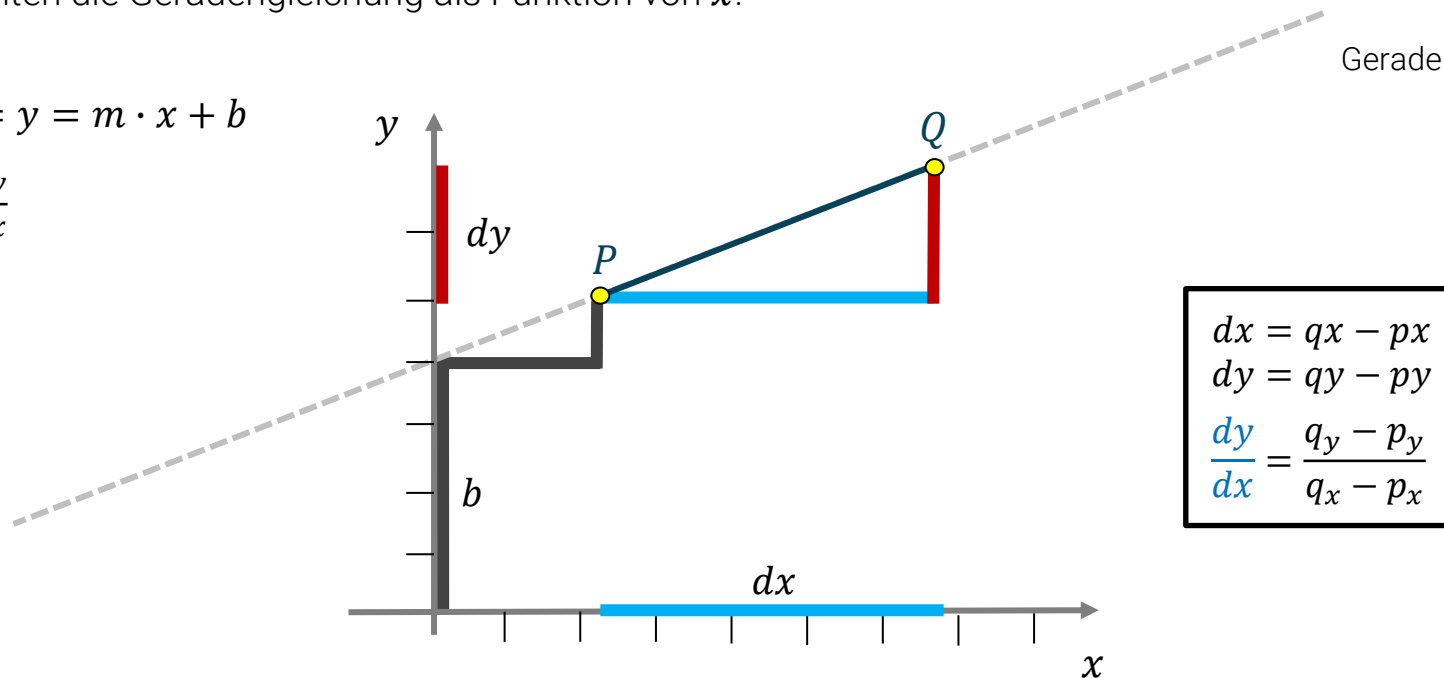
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = ?$$



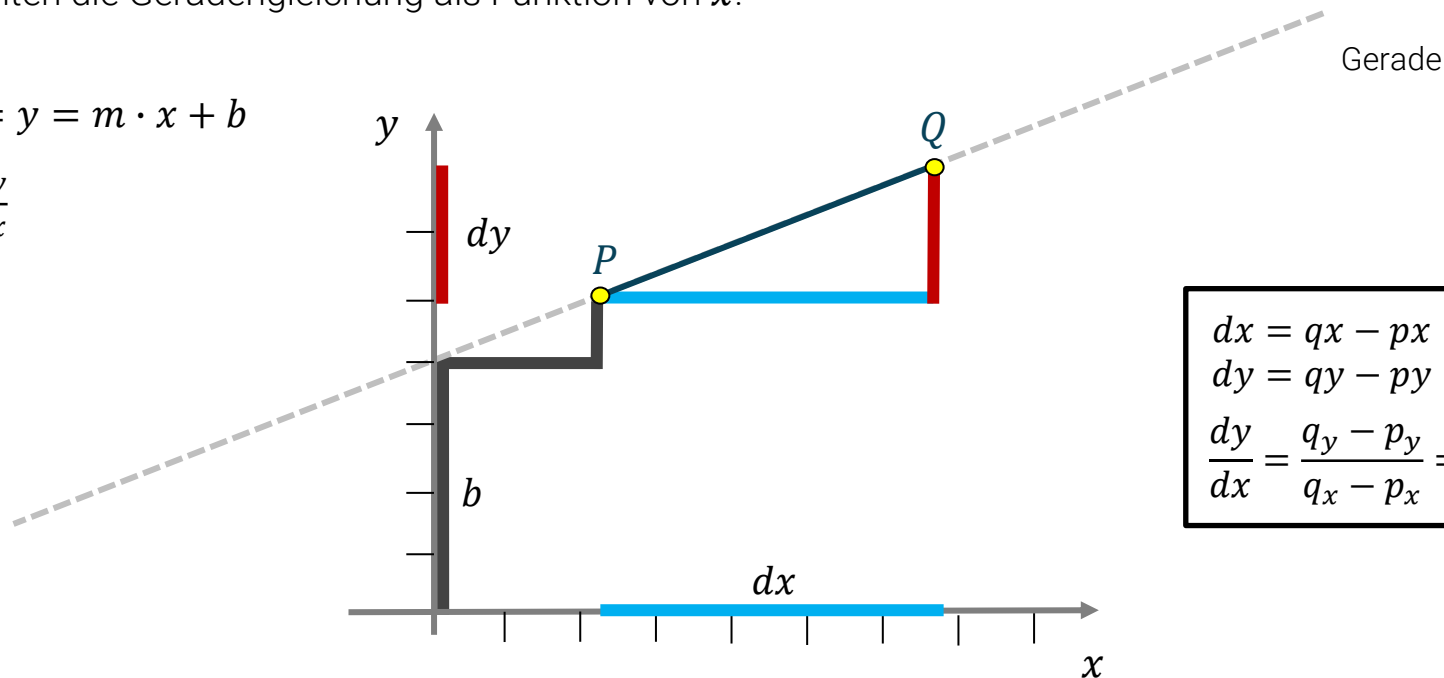
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = ?$$



$$dx = qx - px$$

$$dy = qy - py$$

$$\frac{dy}{dx} = \frac{q_y - p_y}{q_x - p_x} = \frac{p_y - b}{p_x - 0}$$

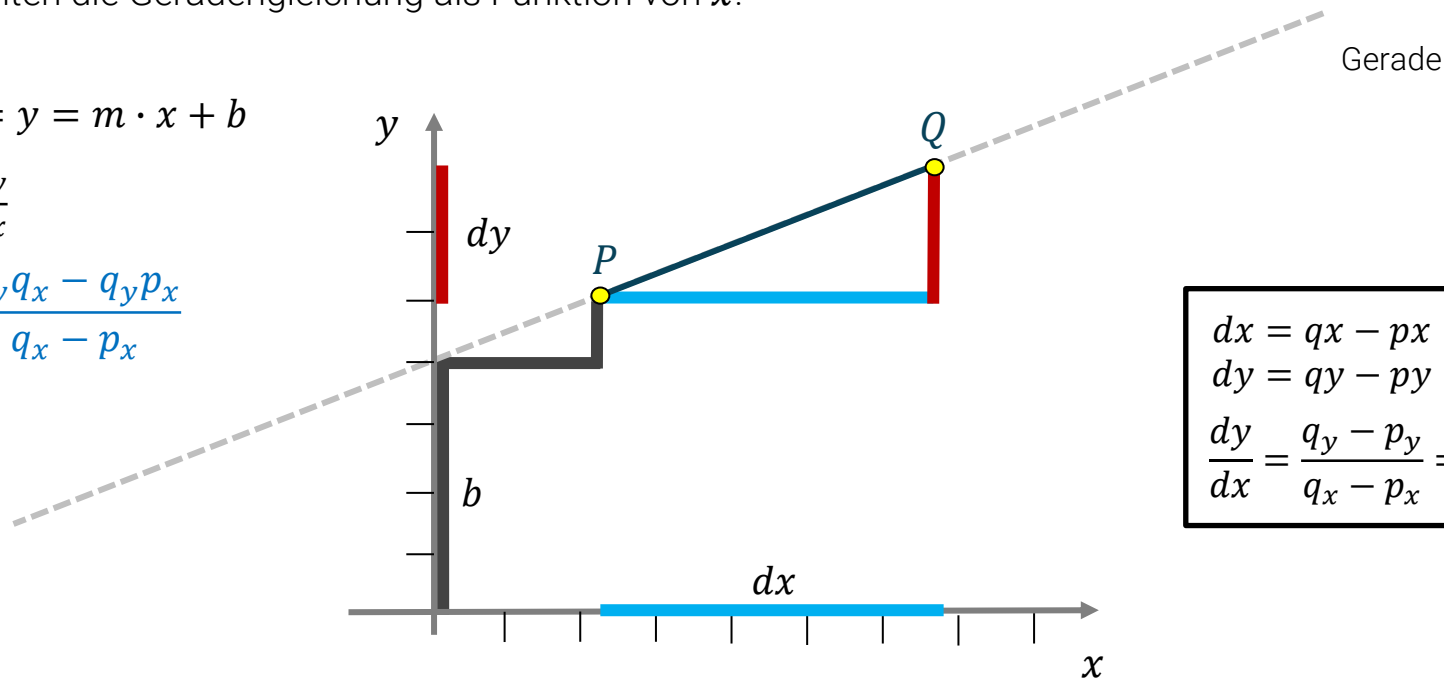
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = \frac{p_y q_x - q_y p_x}{q_x - p_x}$$

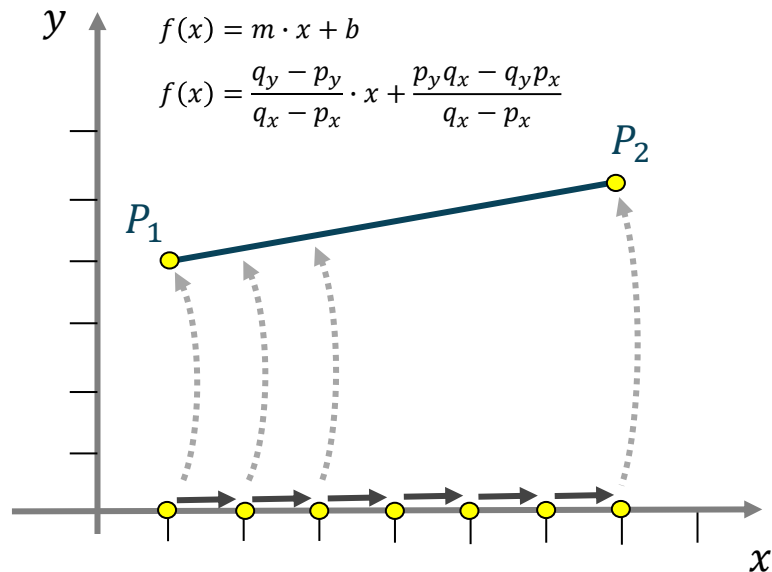


$$\begin{aligned} dx &= qx - px \\ dy &= qy - py \\ \frac{dy}{dx} &= \frac{q_y - p_y}{q_x - p_x} = \frac{p_y - b}{p_x - 0} \end{aligned}$$

Variante 2: Geradengleichung als Funktion

Linie zeichnen - Version 2

Schauen wir uns die Implementierung der zweiten Variante des Linienzeichnens über die [Geradengleichung als Funktion](#) an.



```
void drawLineVersion2(int x1, int y1, int x2, int y2) {  
    // garantiert Linie von links nach rechts fordern  
    if (x2 < x1) {  
        drawLineVersion2(x2, y2, x1, y1);  
    } else {  
        double m = (double) (y2 - y1) / (double) (x2 - x1);  
        double b = (double) (y1 * x2 - y2 * x1) / (double) (x2 - x1);  
  
        for (int x = x1; x <= x2; x++)  
            setPixel(x, m * x + b);  
    }  
}
```

Wir haben die [Gleitkommaarithmetik um die Hälfte reduziert](#). Allerdings haben wir auch hier noch möglicherweise unschöne Darstellungen.

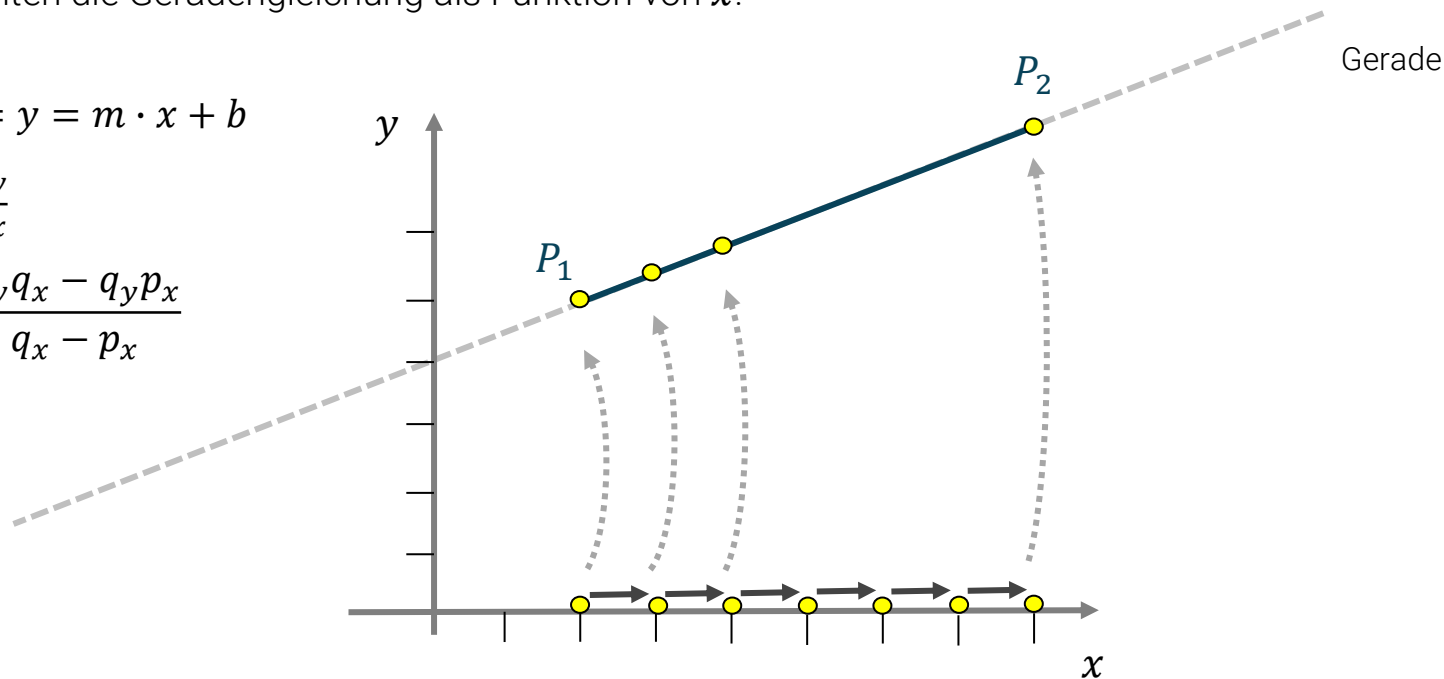
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = \frac{p_y q_x - q_y p_x}{q_x - p_x}$$





Die Schwäche zeigt sich bei größeren Steigungen.

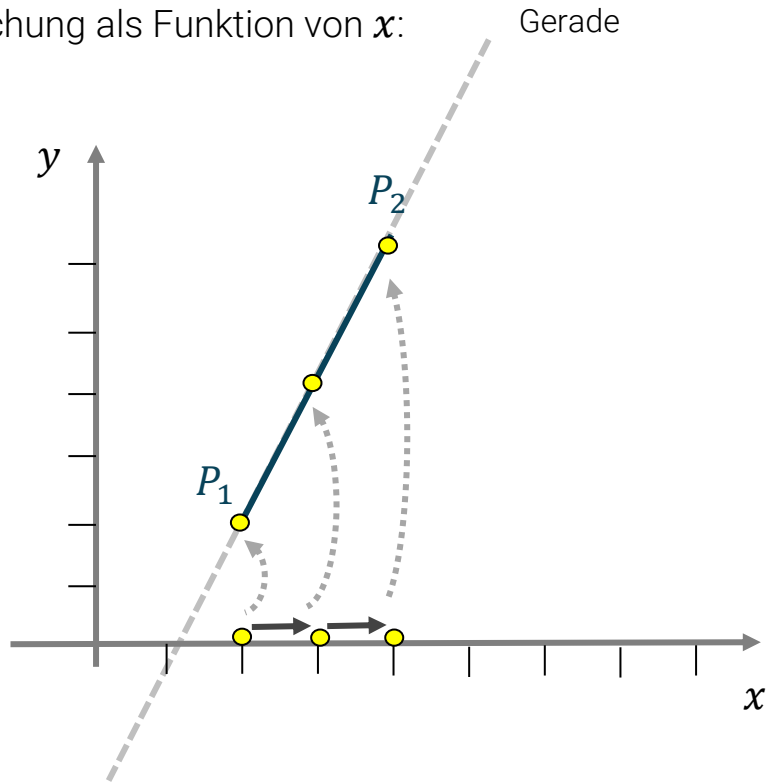
Geradengleichung als Funktion

Wir betrachten die Geradengleichung als Funktion von x :

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

$$b = \frac{p_y q_x - q_y p_x}{q_x - p_x}$$





Wie lässt sich diese Problematik
einfach lösen?

Geradengleichung als Funktion

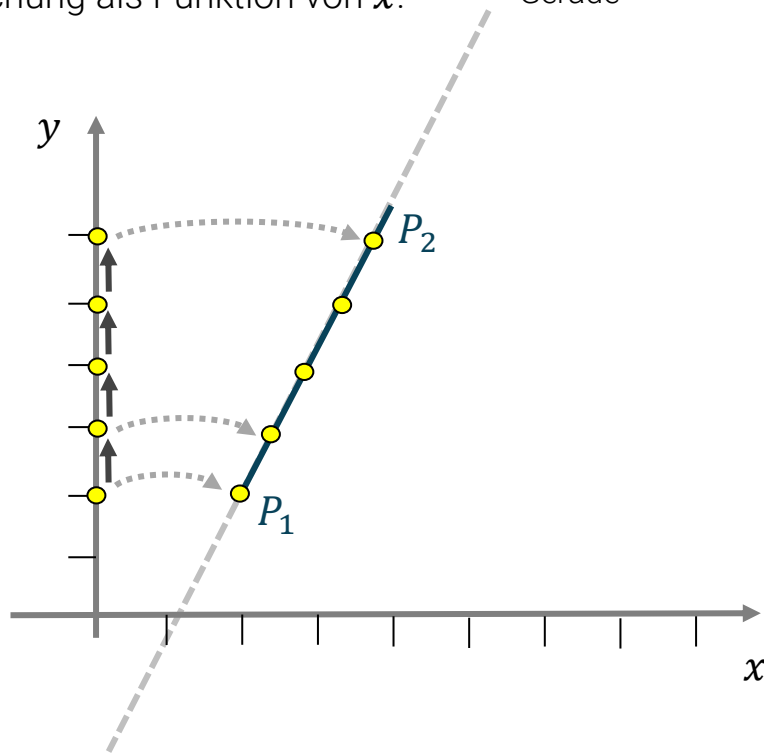
Wir betrachten die Geradengleichung als Funktion von x :

Gerade

$$f(x) = y = m \cdot x + b$$

$$m = \frac{dy}{dx}$$

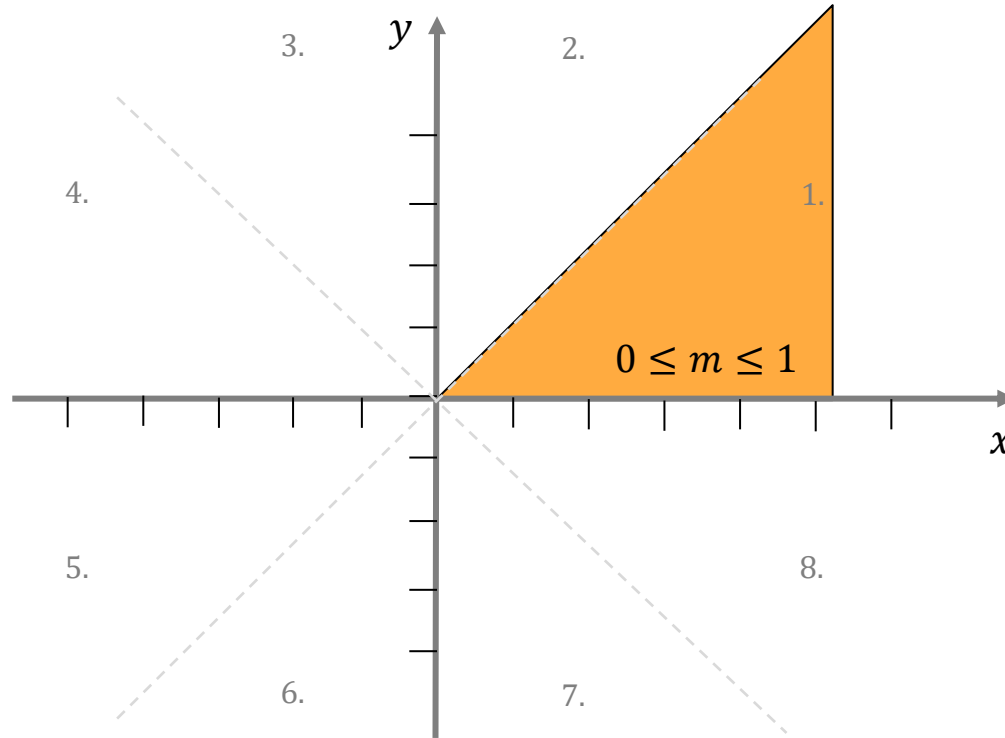
$$b = \frac{p_y q_x - q_y p_x}{q_x - p_x}$$



$$f(y) = x = m \cdot y + b$$

Oktanten und Steigungen

Schauen wir uns die acht **Oktanten** und die Steigungen an. Wir konzentrieren uns zunächst auf Oktant 1.



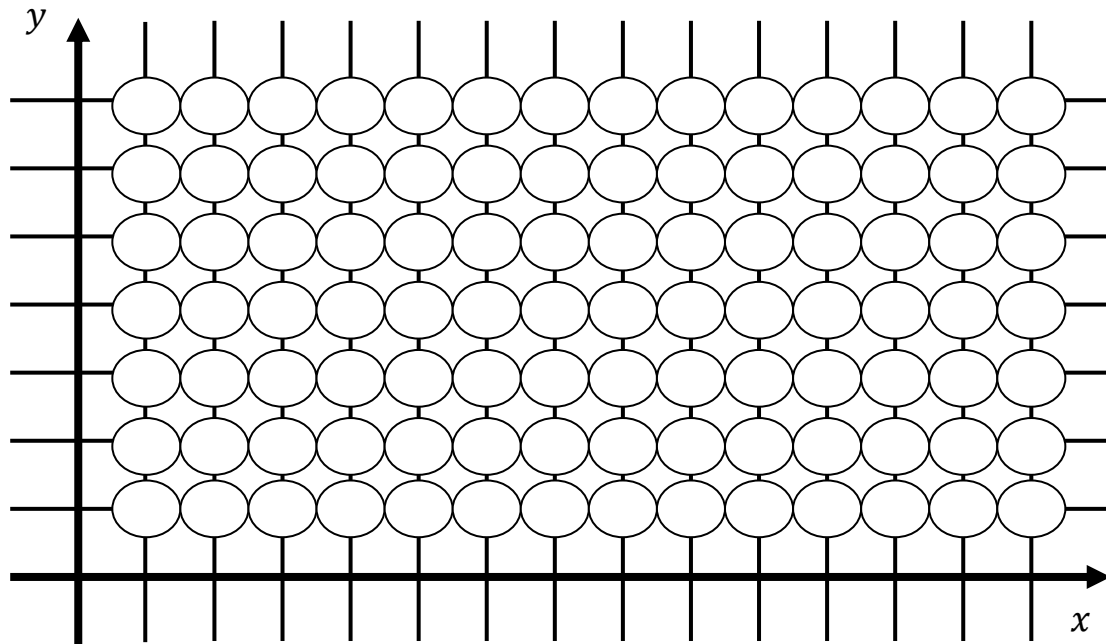
Später transformieren wir die Linien durch Spiegelung/Vertauschung in den 1. Oktanten



Wenn wir sowieso nur Pixel des Rasters anschalten können, warum bleiben x und y nicht ganzzahlig?

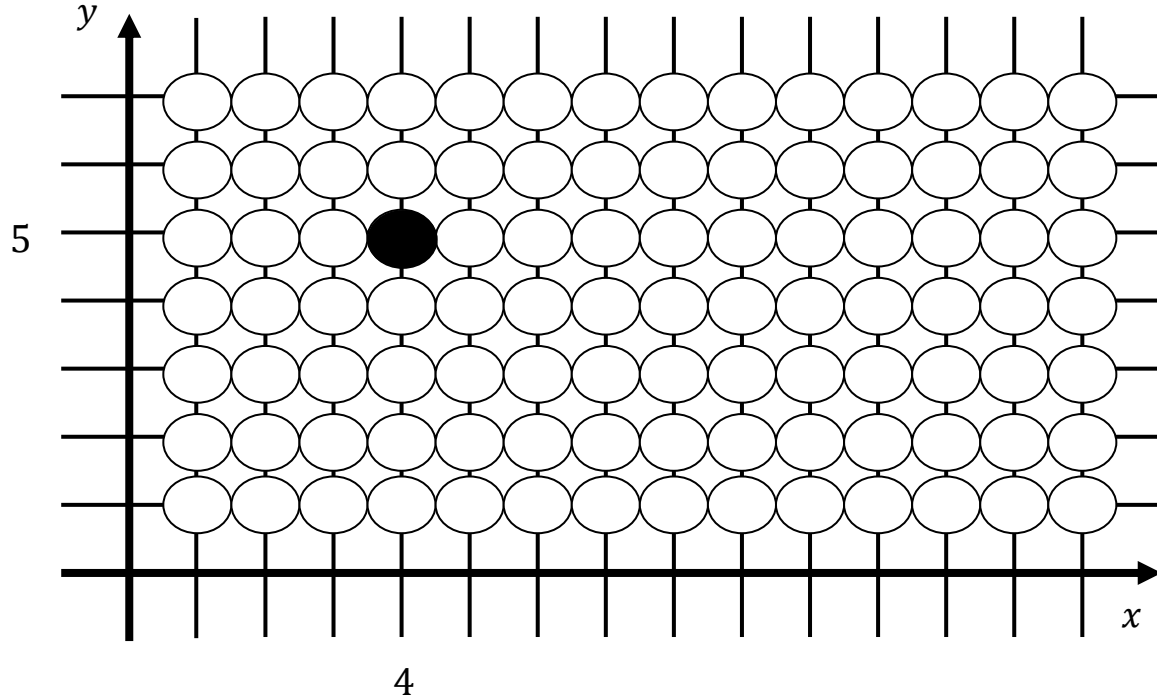
Koordinatenraster

Das zur Verfügung stehende Pixelraster ist ganzzahlig. Hier können wir wie gehabt Pixel an- oder ausschalten.



setPixel(x, y)

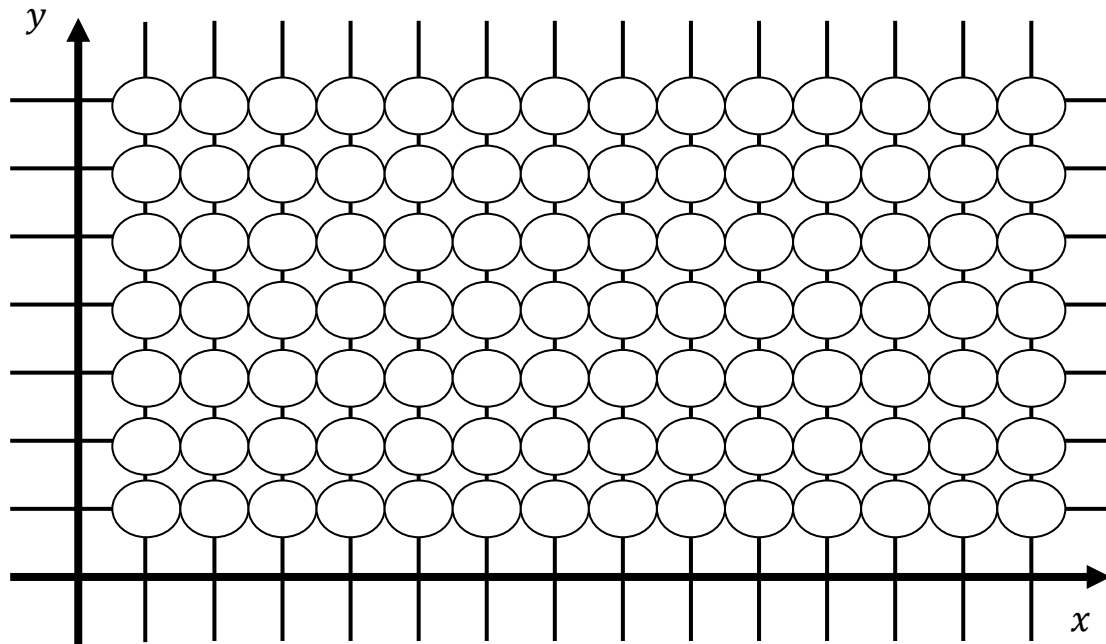
Das zur Verfügung stehende Pixelraster ist ganzzahlig. Hier können wir wie gehabt Pixel an- oder ausschalten.



Variante 3: Algorithmus von Bresenham

Algorithmus von Bresenham

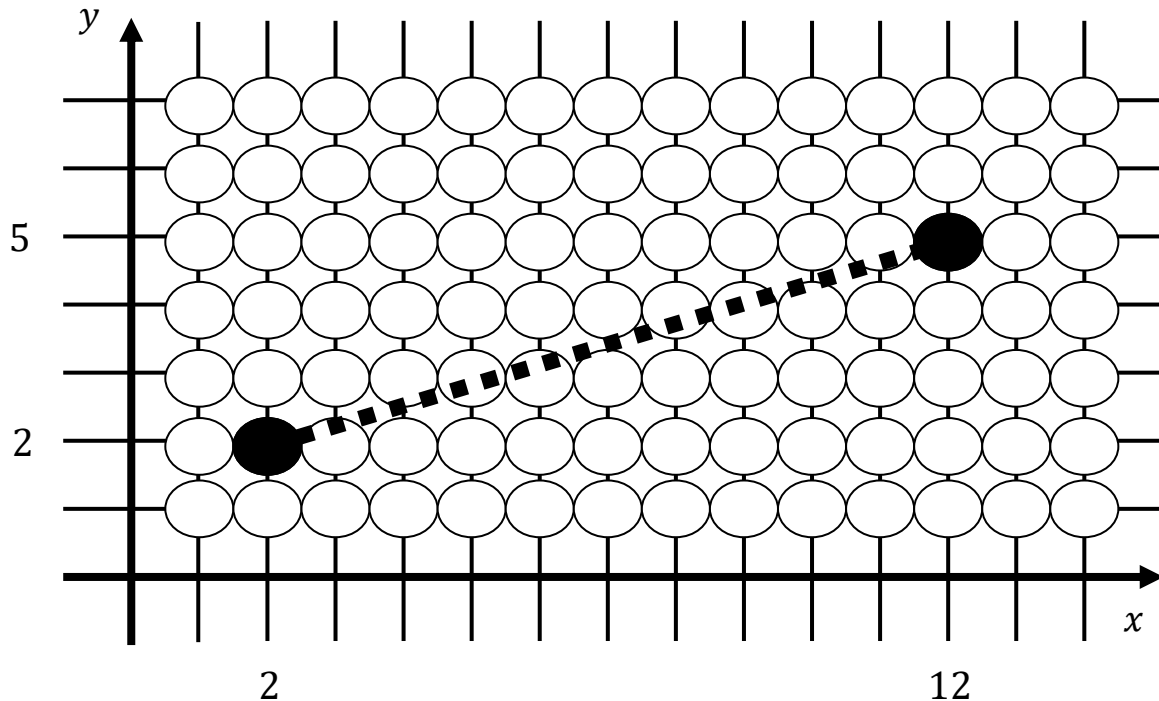
Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an [1].



[1] Bresenham J. E.: "Algorithm for Computer Control of a Digital Plotter", IBM Systems Journal, Vol. 4, No. 1, pp. 25–30, 1965

Algorithmus von Bresenham

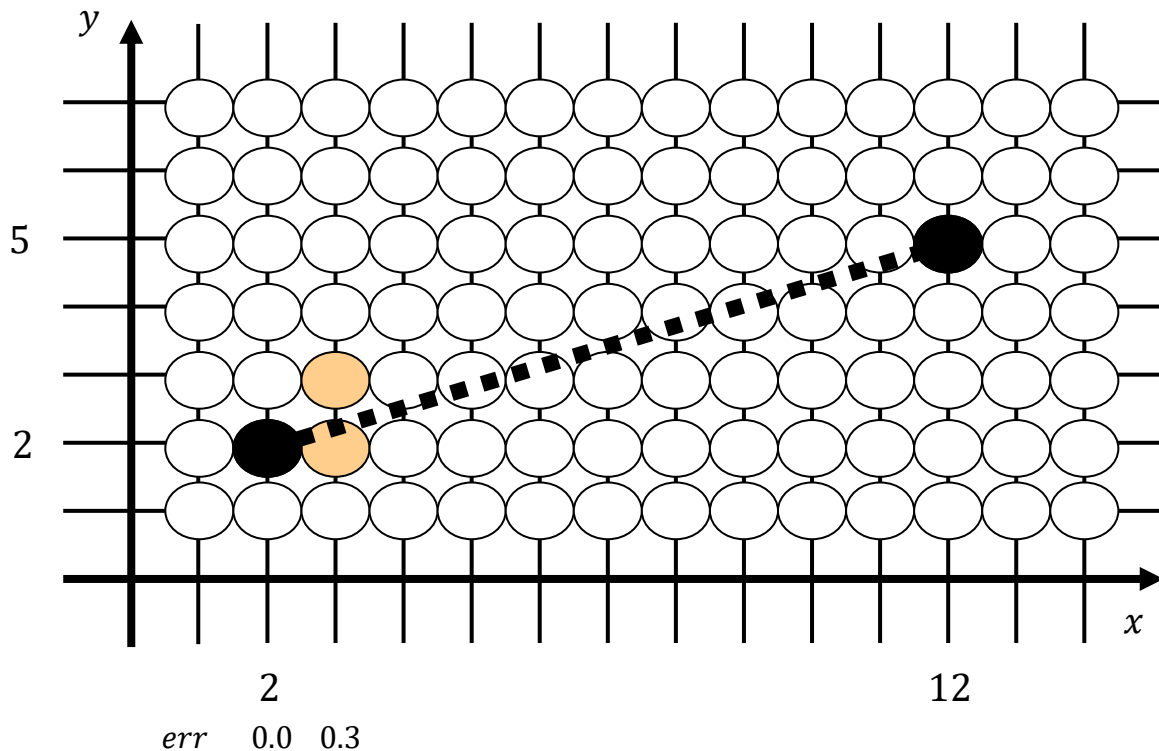
Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.



$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

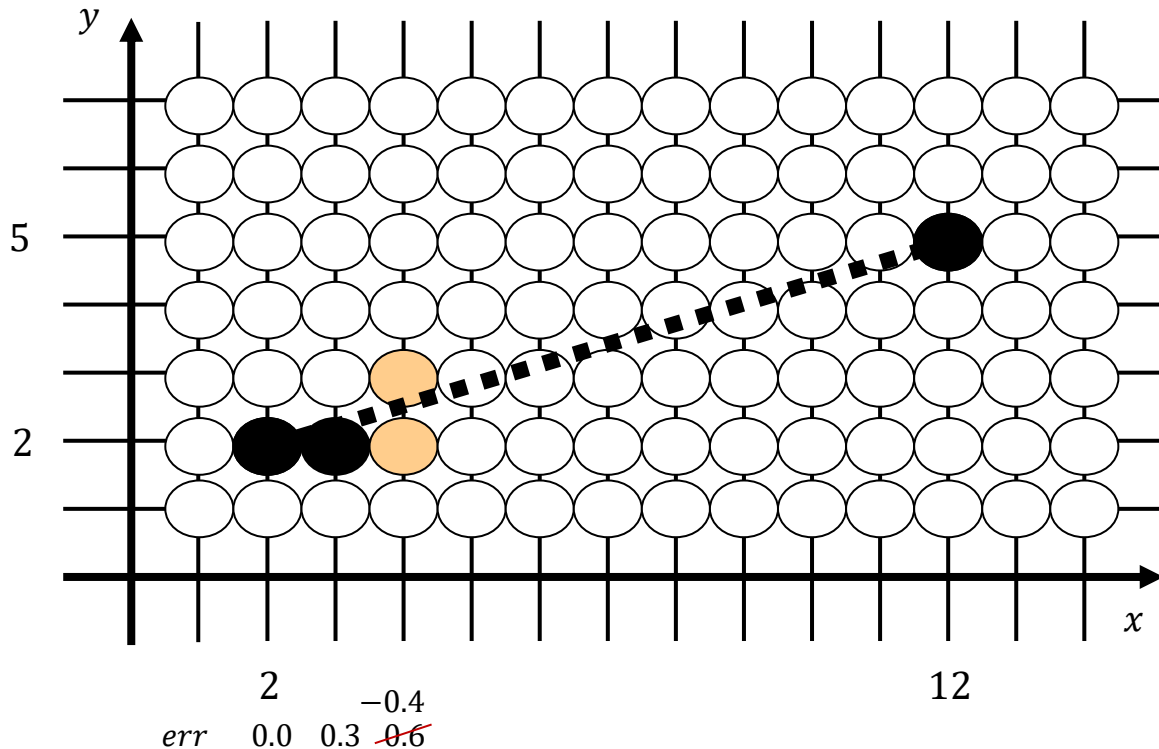


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

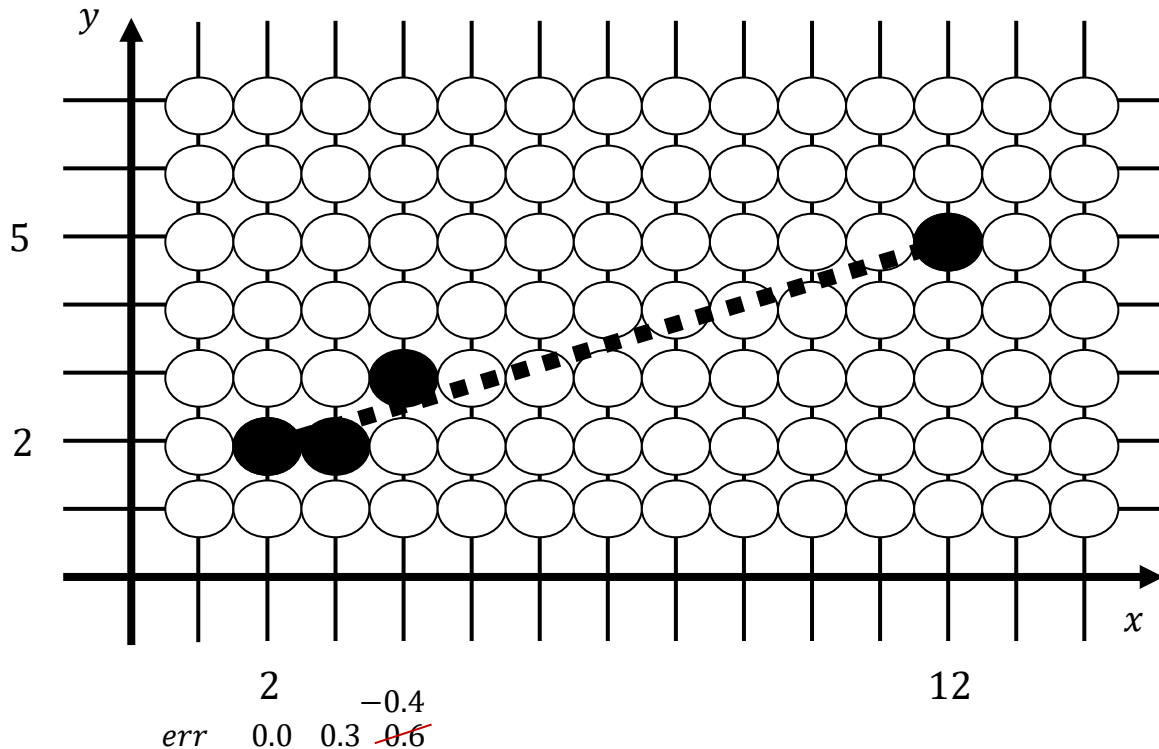


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

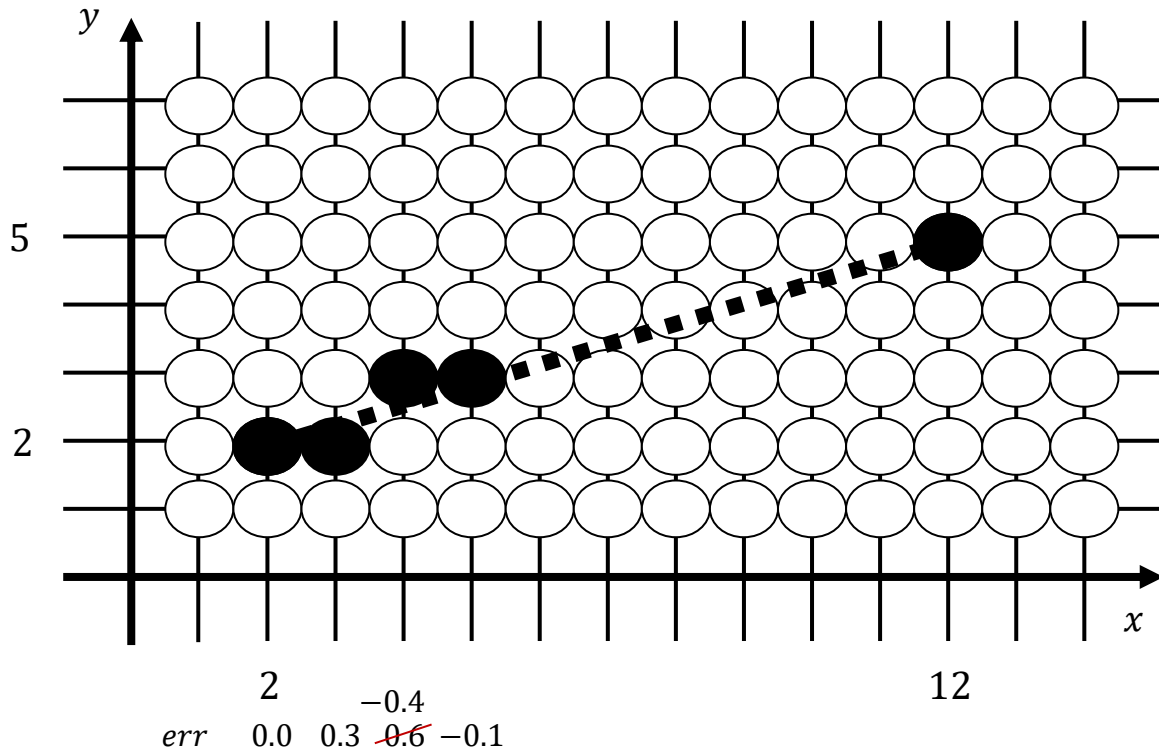


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

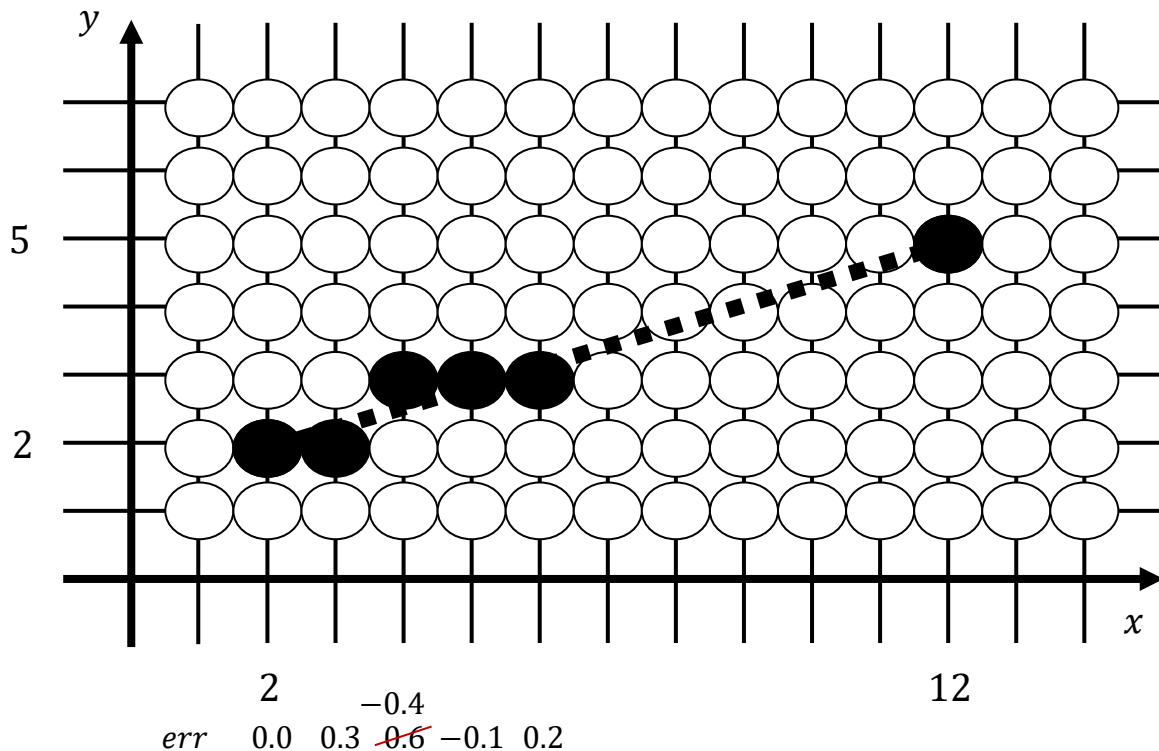


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

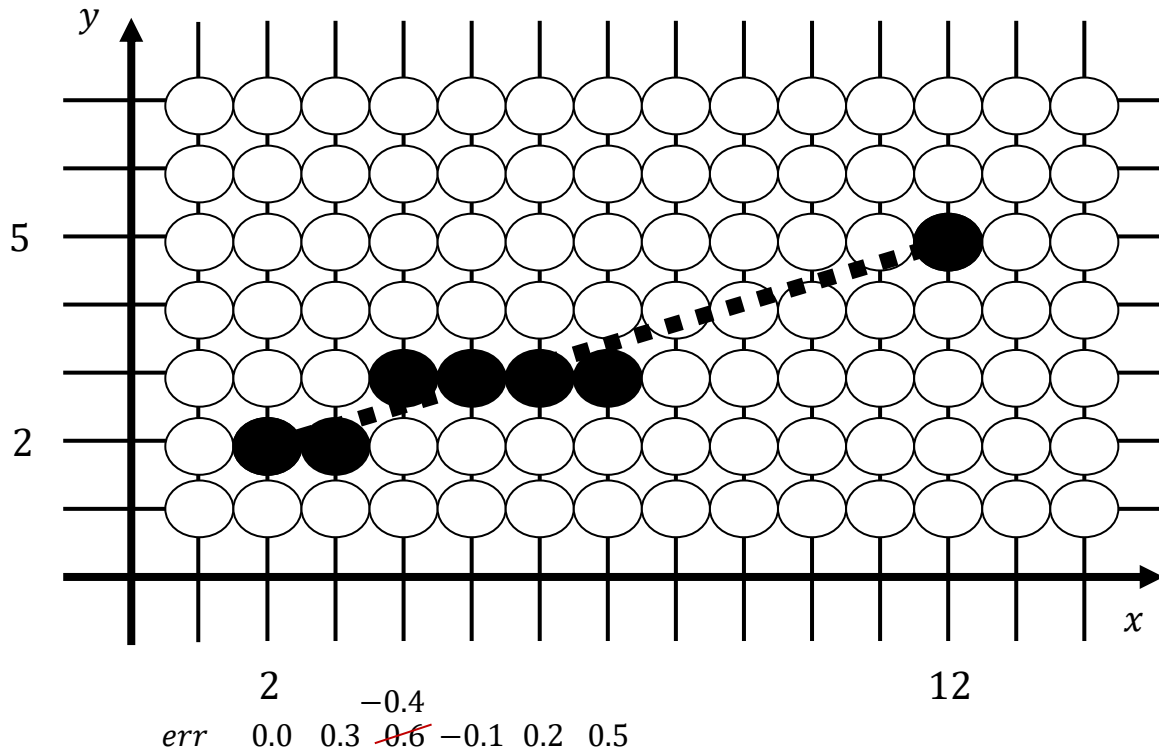


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

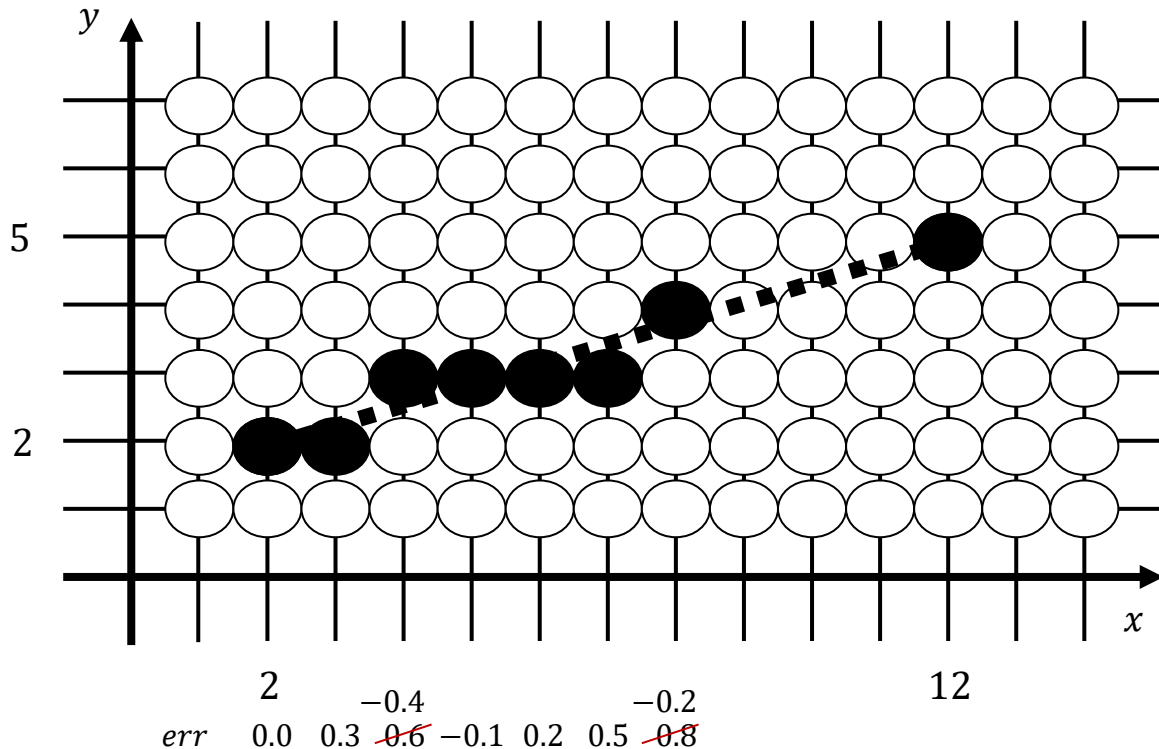


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

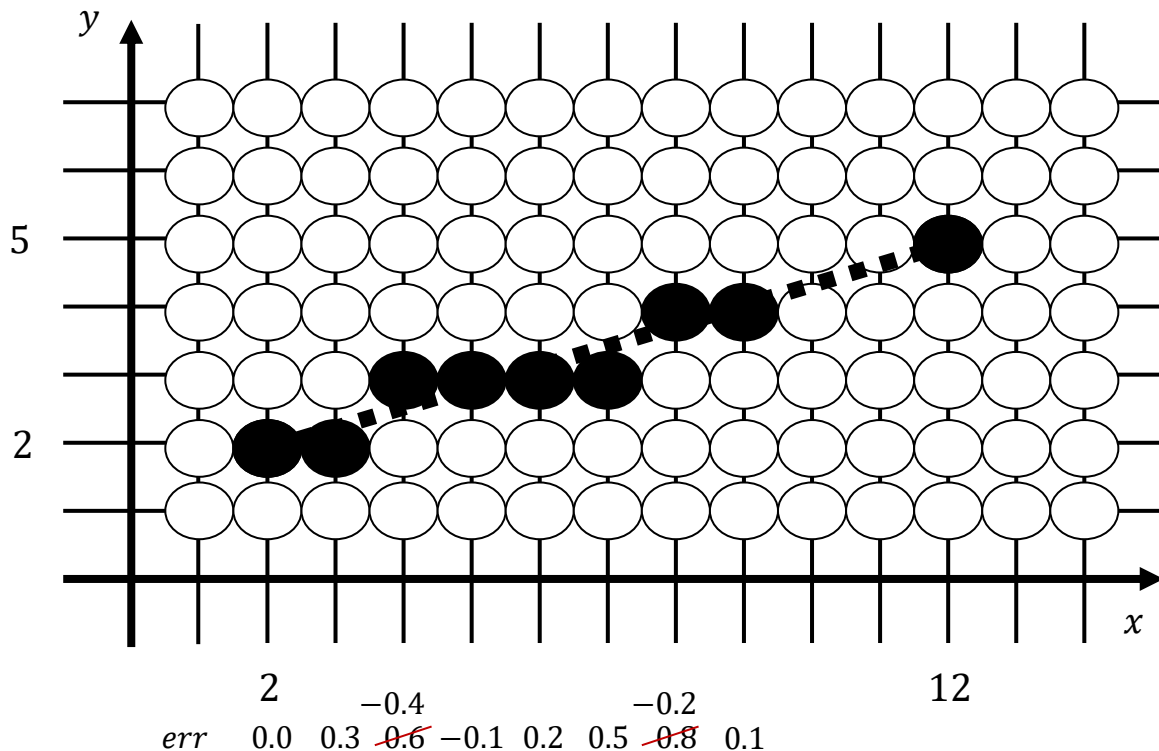


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

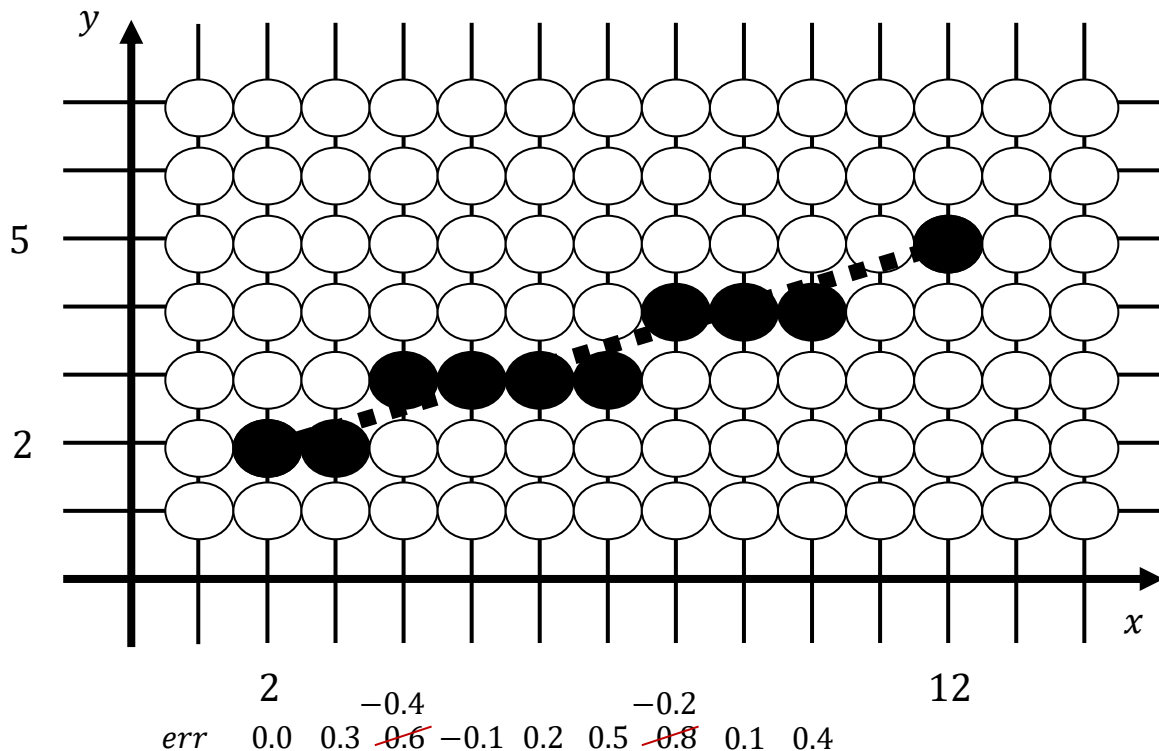


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

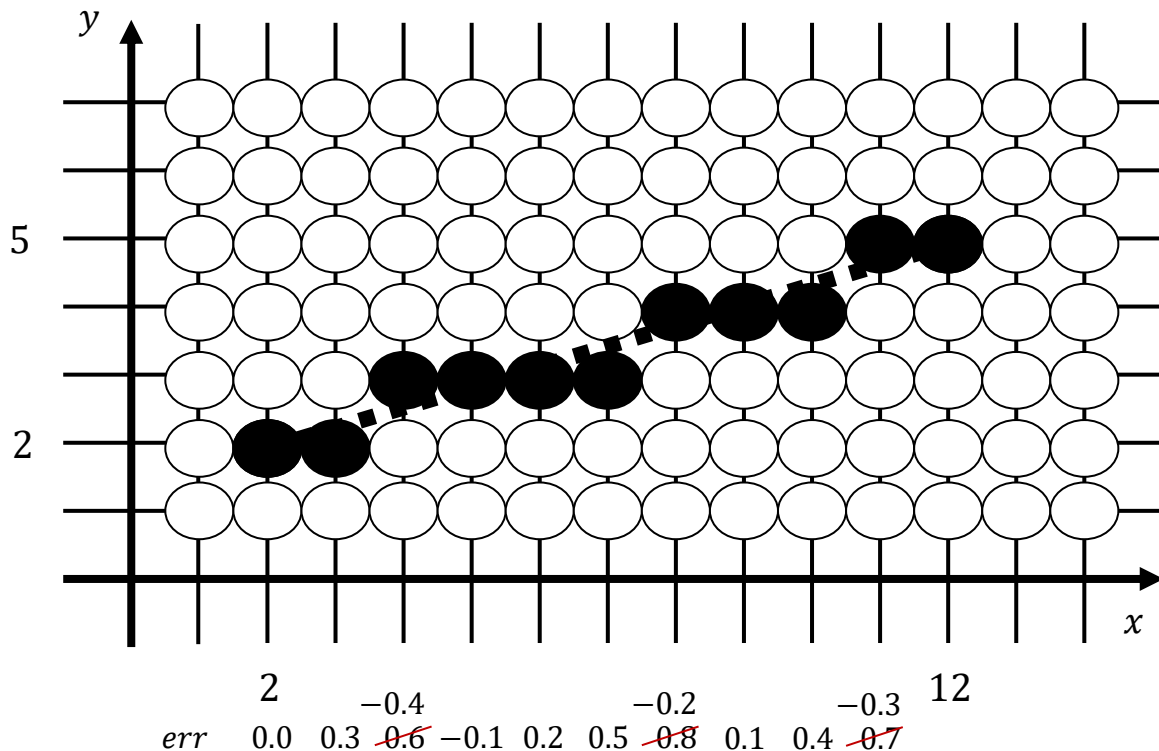


$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Algorithmus von Bresenham

Schauen wir uns schrittweise den Algorithmus zum Zeichnen einer Linie nach Bresenham an.

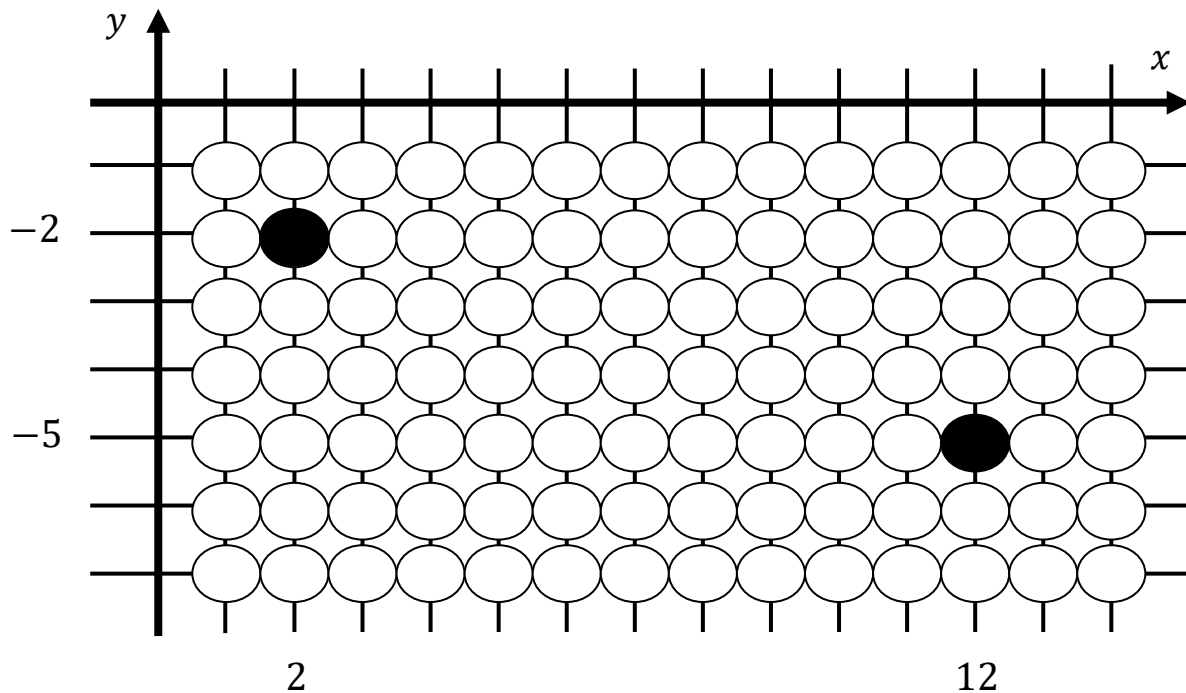




Eigene Übungsaufgaben auf einem Karopapier

Algorithmus von Bresenham

Bitte überprüft doch mal, ob sich das vorgestellte Vorgehen von Bresenham sich auch für diese Linie eignet.



Kommen wir zur Implementierung

```

void drawLineVersionBresenham1(int x1, int y1, int x2, int y2) {
    int dx    = x2 - x1;           // Weg in x
    int dy    = y2 - y1;           // Weg in y
    double m   = (double)dy/dx;
    double err = 0.0;
    int y      = y1;

    for (int x=x1; x<=x2; x++) {
        setPixel(x, y);
        err += m;
        if (err>0.5) {
            y++;
            err-=1;
        }
    }
}

```



$$m = \frac{dy}{dx}$$

$$m_2 = m \cdot 2 dx = 2 dy$$

$$0.5 \cdot 2 dx = dx$$

$$1 \cdot 2 dx = 2 dx$$

Linie zeichnen - Bresenham 1

Schauen wir uns die Implementierung der ersten Variante des Linienzeichnens von Bresenham an.

```
void drawLineVersionBresenham1(int x1, int y1, int x2, int y2) {  
    int dx      = x2 - x1;           // Weg in x  
    int dy      = y2 - y1;           // Weg in y  
    double m    = (double)dy/dx;  
    double err  = 0.0;  
    int y       = y1;  
  
    for (int x=x1; x<=x2; x++) {  
        setPixel(x, y);  
        err += m;  
        if (err>0.5) {  
            y++;  
            err-=1;  
        }  
    }  
}
```

Jetzt wollen wir noch die **Steigung** und den **Fehler** ganzzahlig realisieren. Dazu müssen wir die Steigung mit einem geeigneten Faktor multiplizieren:

$$m_{alt} = \frac{dy}{dx}$$

$$m_{neu} = m_{alt} \cdot 2dx = \frac{dy}{dx} \cdot 2dx = 2dy$$

Die Parameter **x** und **y** sind jetzt beide **ganzzahlig**. So weit so gut.

Linie zeichnen - Bresenham 2

Schauen wir uns die Implementierung der zweiten Variante des Linienzeichnens von Bresenham an.

```
void drawLineVersionBresenham2(int x1, int y1, int x2, int y2) {
    int dx  = x2 - x1;           // Weg in x
    int dy  = y2 - y1;           // Weg in y
    int m   = 2*dy;
    int err = 0;
    int y   = y1;

    for (int x=x1; x<=x2; x++) {
        setPixel(x, y);
        err += m;
        if (err>dx) {
            y++;
            err-=2*dx;
        }
    }
}
```

Eine kleine Optimierung können wir erreichen, indem wir den Vergleich mit **0** (der direkt als Maschinenbefehl vorliegt) durchführen und den Fehler entsprechend um dx nach unten verschieben.

Die Parameter **x** und **y** sind jetzt beide ganzzahlig. Auch **Steigung** und **Fehler** sind jetzt ganzzahlig.

Linie zeichnen - Bresenham 3

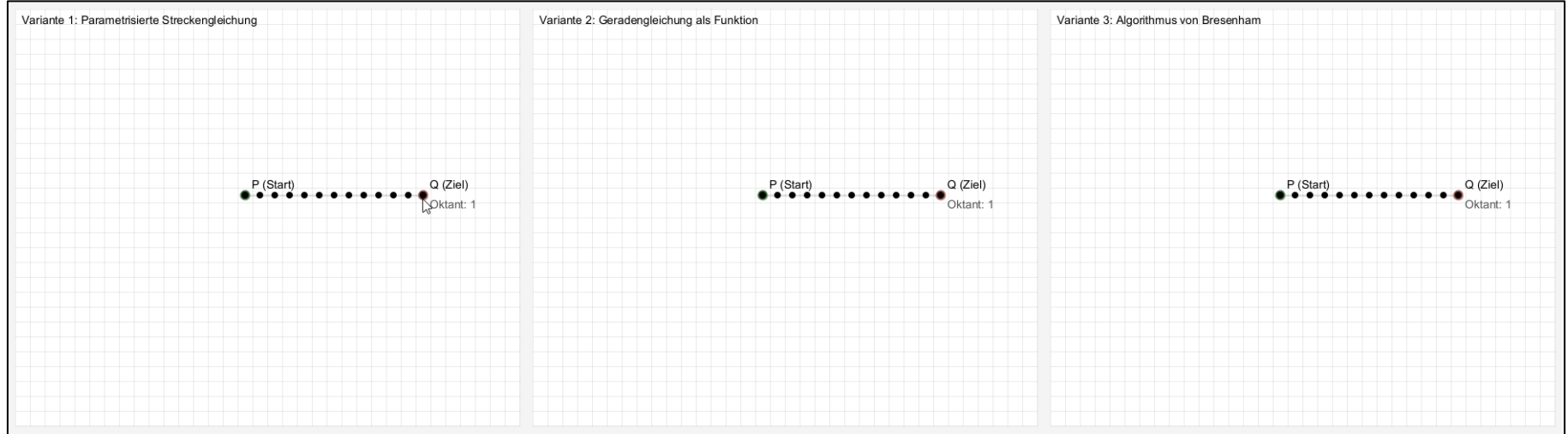
Schauen wir uns die Implementierung der dritten Variante des Linienzeichnens von Bresenham an.

```
void drawLineVersionBresenham3(int x1, int y1, int x2, int y2) {  
    int dx      = x2 - x1;           // Weg in x  
    int dy      = y2 - y1;           // Weg in y  
    int m      = 2*dy;  
    int err     = -dx;  
    int schritt = 2*dx;  
    int y       = y1;  
  
    for (int x=x1; x<=x2; x++) {  
        setPixel(x, y);  
        err += m;  
        if (err>0) {  
            y++;  
            err-=schritt;  
        }  
    }  
}
```

Jetzt haben wir die finale Version.

Vergleich der drei Varianten

Schauen wir uns hier noch einmal übersichtlich einen Vergleich der Methoden zur Linienkonstruktion an.



Parametrisierte Streckengleichung

- identischer Linienvorlauf
- x und y Gleitkommazahlen

Geradengleichung als Funktion

- identischer Linienvorlauf
- x oder y Gleitkommazahlen

Algorithmus von Bresenham

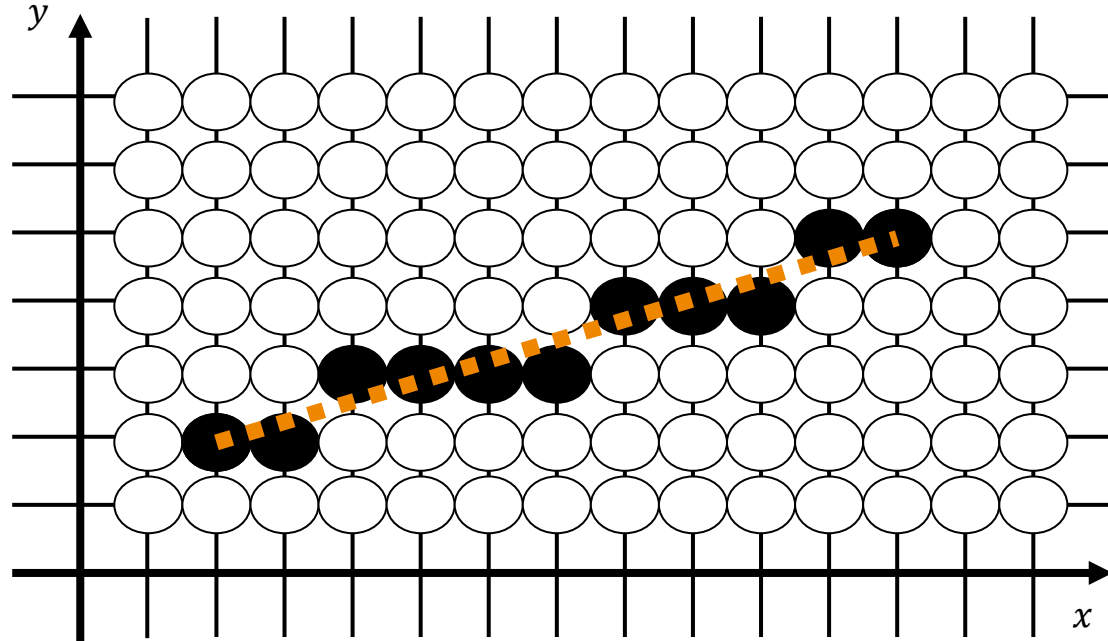
- identischer Linienvorlauf
- x und y Ganzzahlen



Bei aller Mühe, die Effizienz zu steigern,
haben wir trotzdem immer noch einen unschönen
Effekt ...

Aliasing-Effekt

Egal, welche Linienvariante wir einsetzen, der entstehende Treppeneffekt ist unschön:

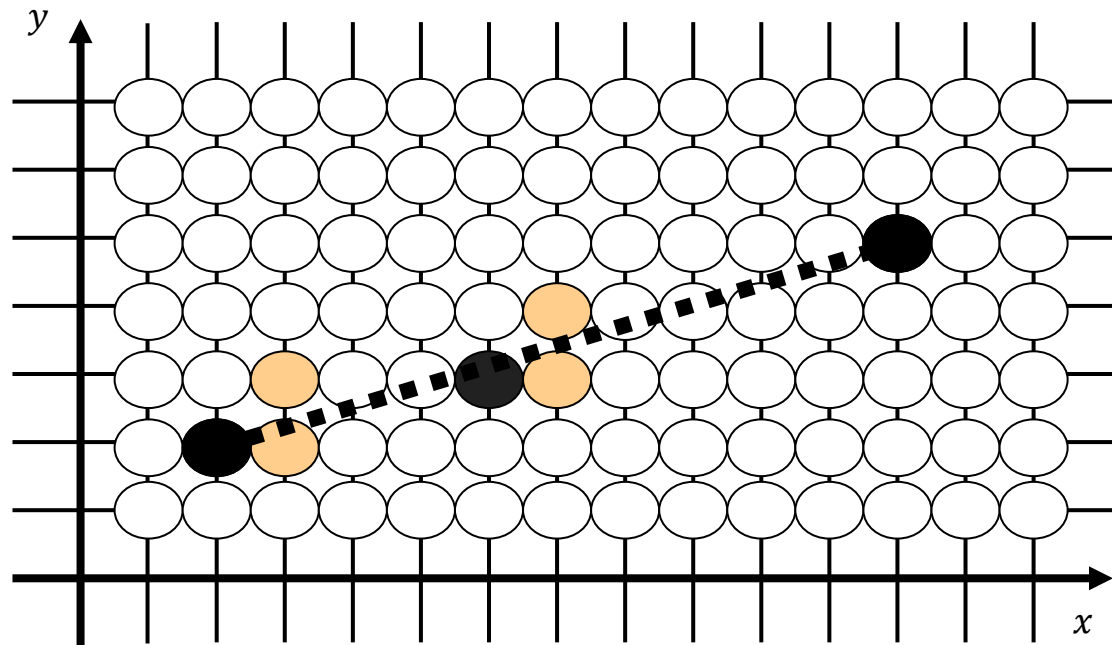




Gibt es Vorschläge?
Was könnten wir machen?

Interpretation des Fehlers

Wie lässt sich der bei Bresenham ermittelte Fehler interpretieren?



$$m = \frac{dy}{dx} = \frac{3}{10} = 0.3$$

$$err = y_{ideal} - y_{real}$$

Wir treffen eine Entscheidung in Bezug zum lokalen Fehler:

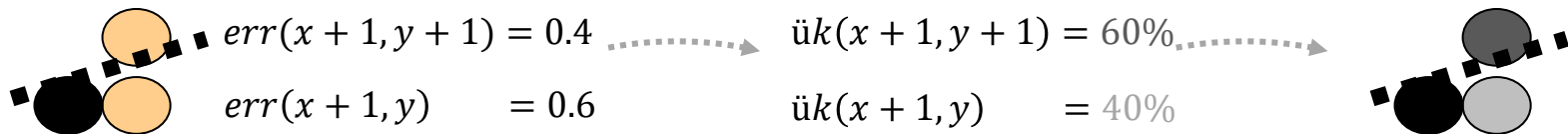
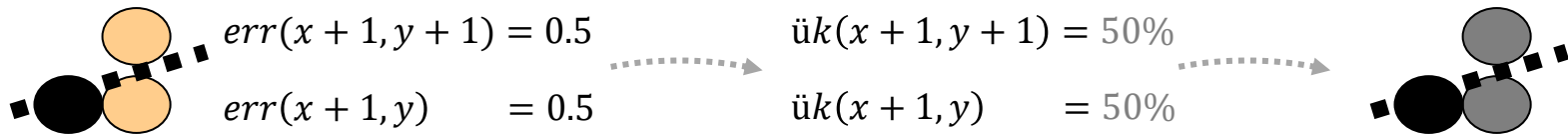
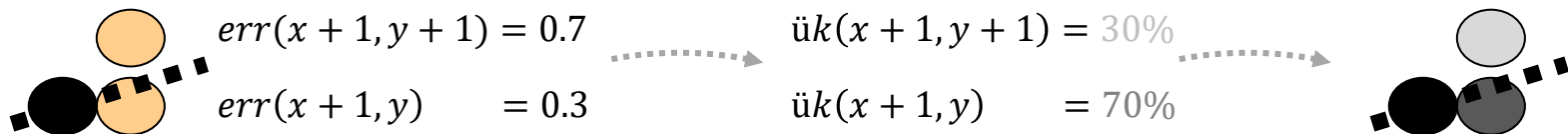
$$\begin{aligned} &err(x+1, y+1) = 0.7 \\ &err(x+1, y) = 0.3 \end{aligned}$$

$$\begin{aligned} &err(x+1, y+1) = 0.5 \\ &err(x+1, y) = 0.5 \end{aligned}$$

err 0.0 0.3

Interpretation des Fehlers

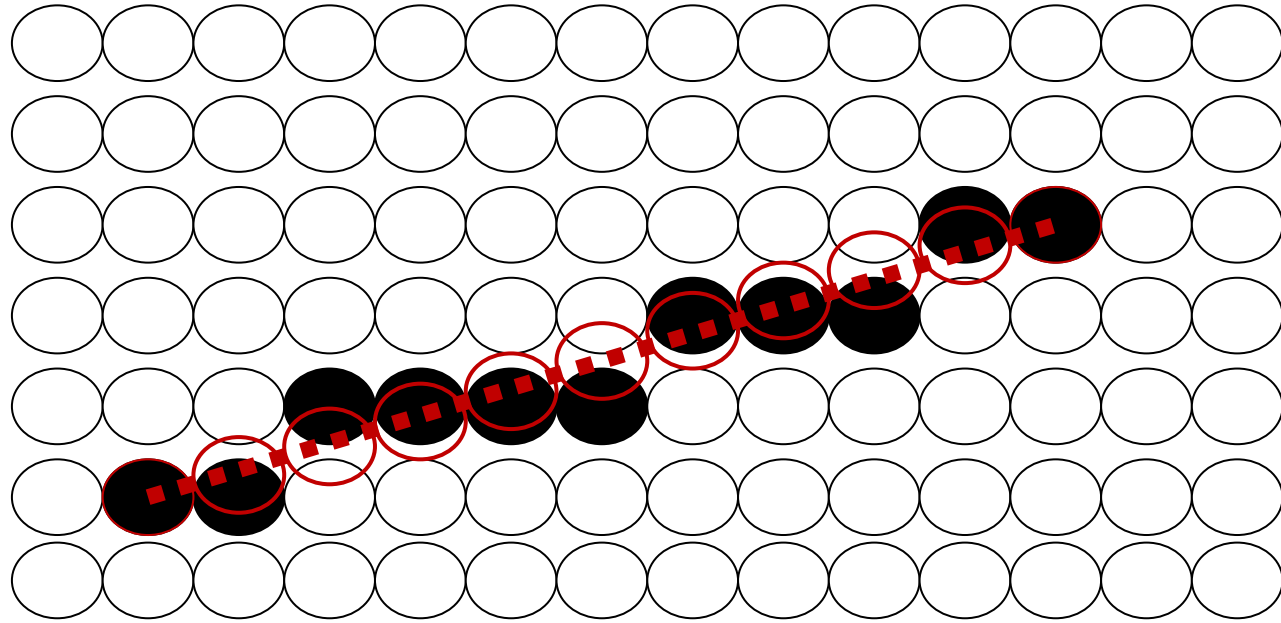
Den Fehler wollen wir jetzt mal anders darstellen. Wir nutzen dafür beide in Frage kommenden Pixel und drücken die Zugehörigkeit des Punktes in Bezug auf die darzustellende Linie aus [1].





Ein Pixel ist eine Fläche. Die Linienhelligkeit entspricht dem Anteil der Pixelfläche, der von der idealen Linie überdeckt wird ([Coverage](#)).

Antialiasing/Kantenglättung

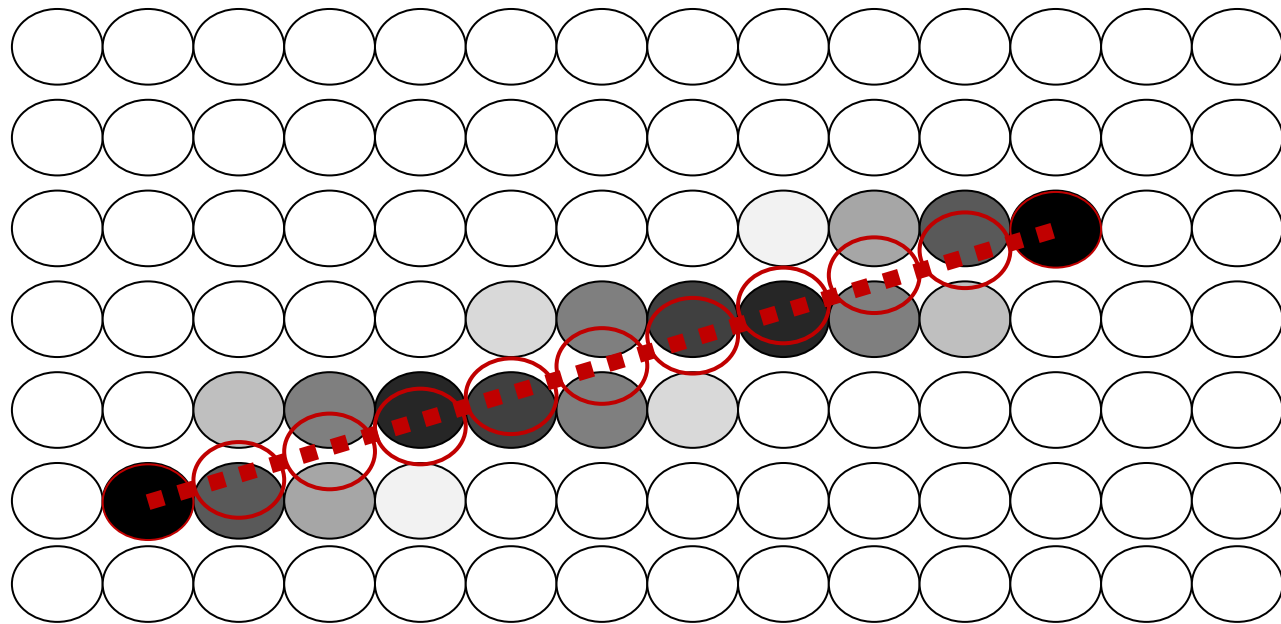


Der entstehende Treppeneffekt ist unschön und kann über eine einfache Idee aus etwas größerer Entfernung abgeschwächt werden.

Überlappungskoeffizienten

						10	40	70	100
				20	50	80	90	60	30
	30	60	90	80	50	20			
100	70	40	10						

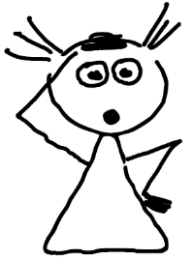
Antialiasing/Kantenglättung



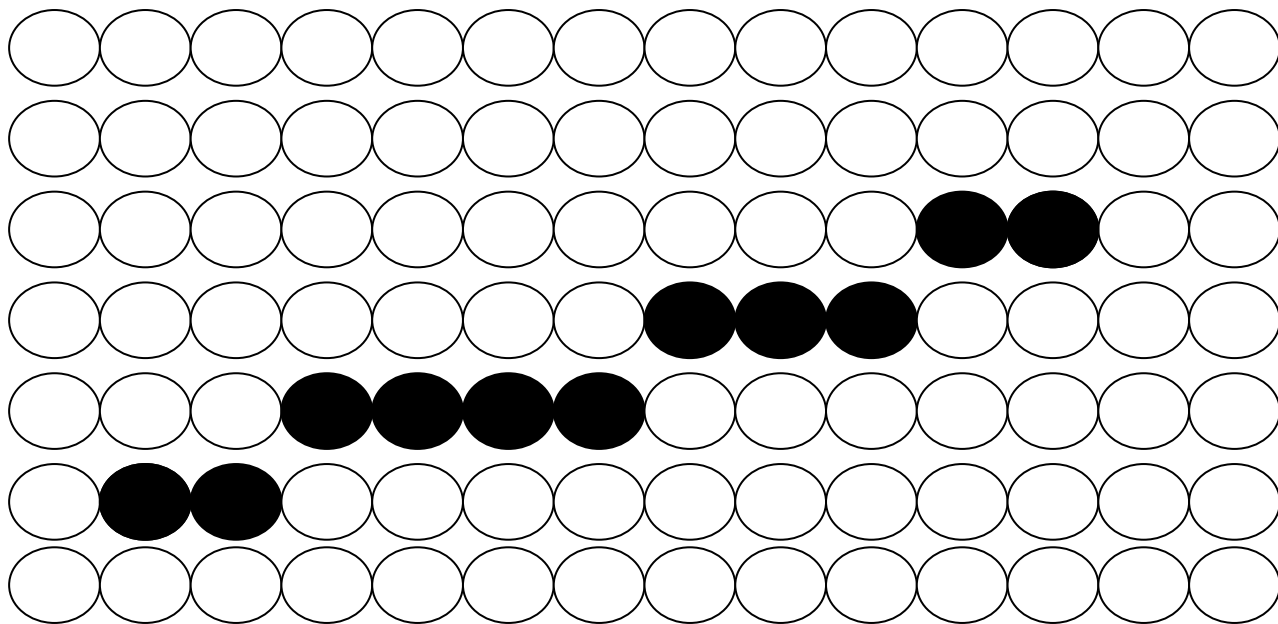
Der entstehende Treppeneffekt ist unschön und kann über eine einfache Idee aus etwas größerer Entfernung abgeschwächt werden.

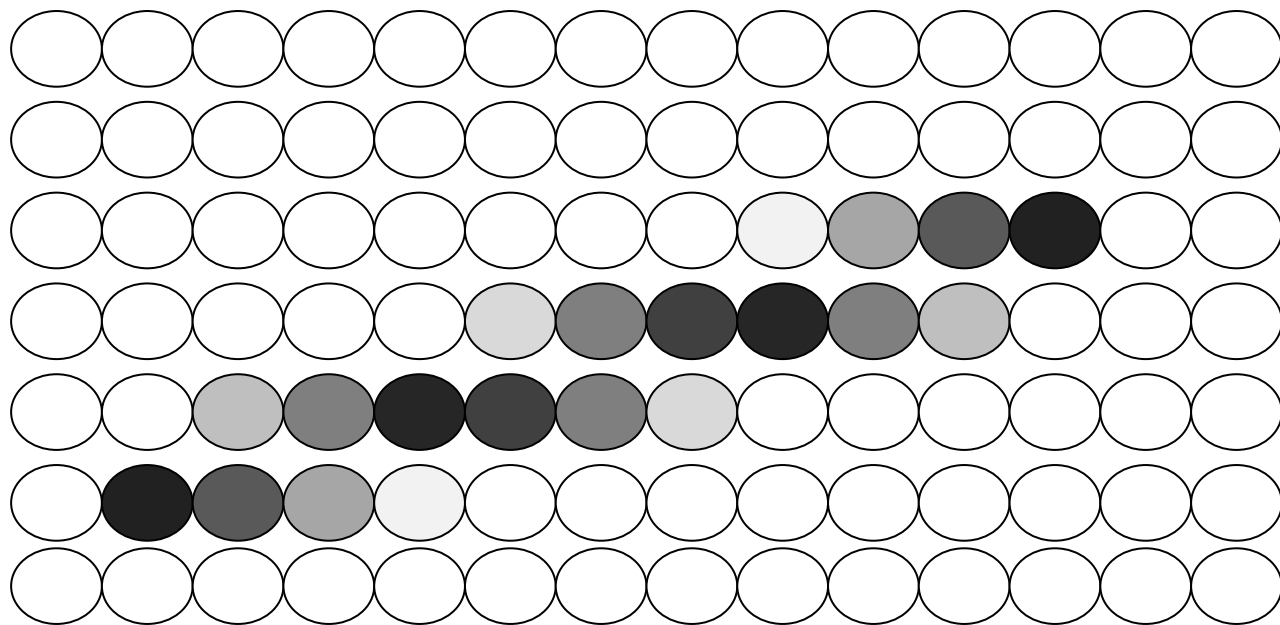
Überlappungskoeffizienten

						10	40	70	100
			20	50	80	90	60	30	
	30	60	90	80	50	20			
100	70	40	10						



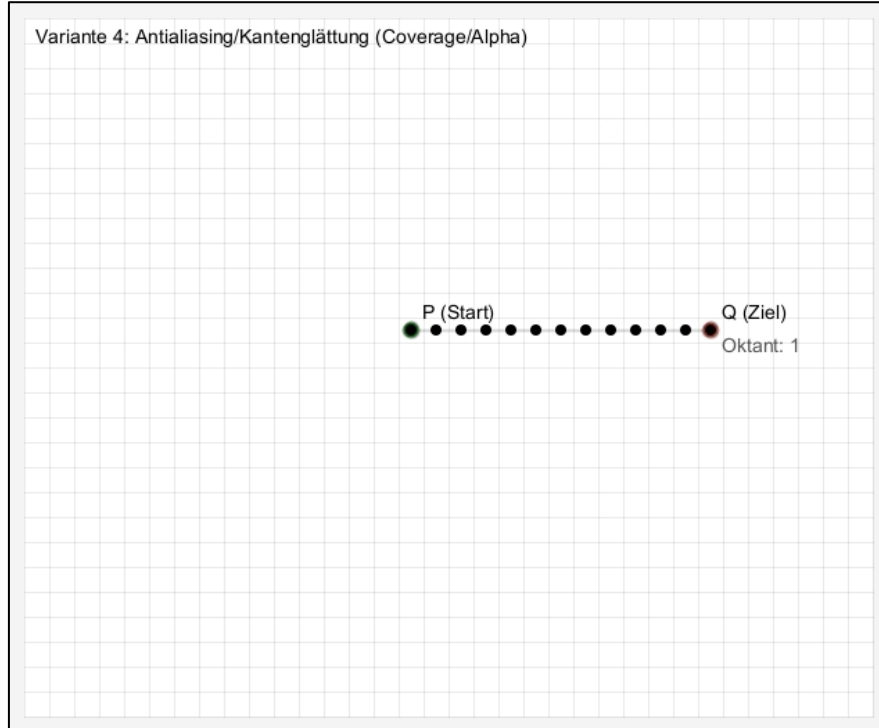
Rücken wir mal etwas nach hinten ...





Linienkonstruktion und Anti-Aliasing

Schauen wir uns hier noch einmal die Darstellung einer Linie mit den Überlappungskoeffizienten an.

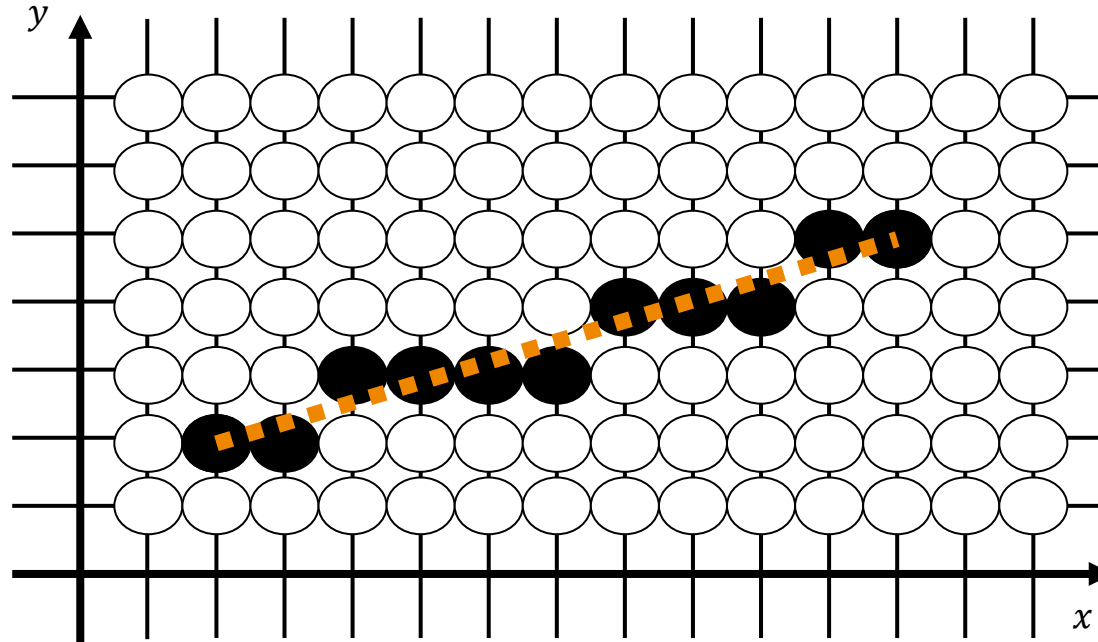




Wie schaut es eigentlich aus, wenn wir dicke Linien zeichnen wollen?

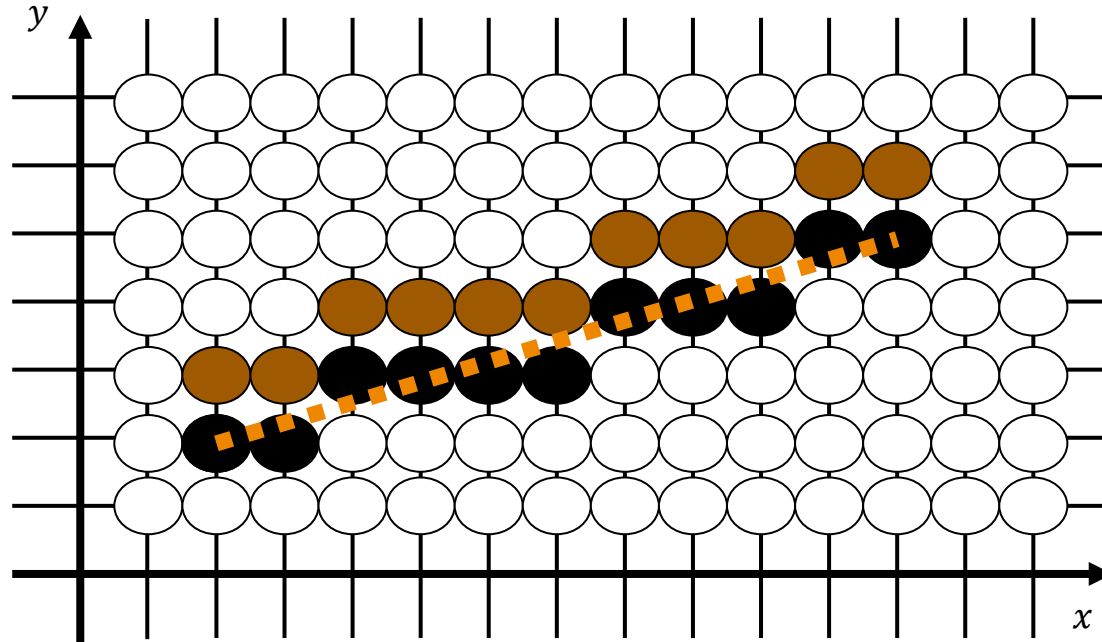
Pixelwiederholungen – Breite=1

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut [1]:



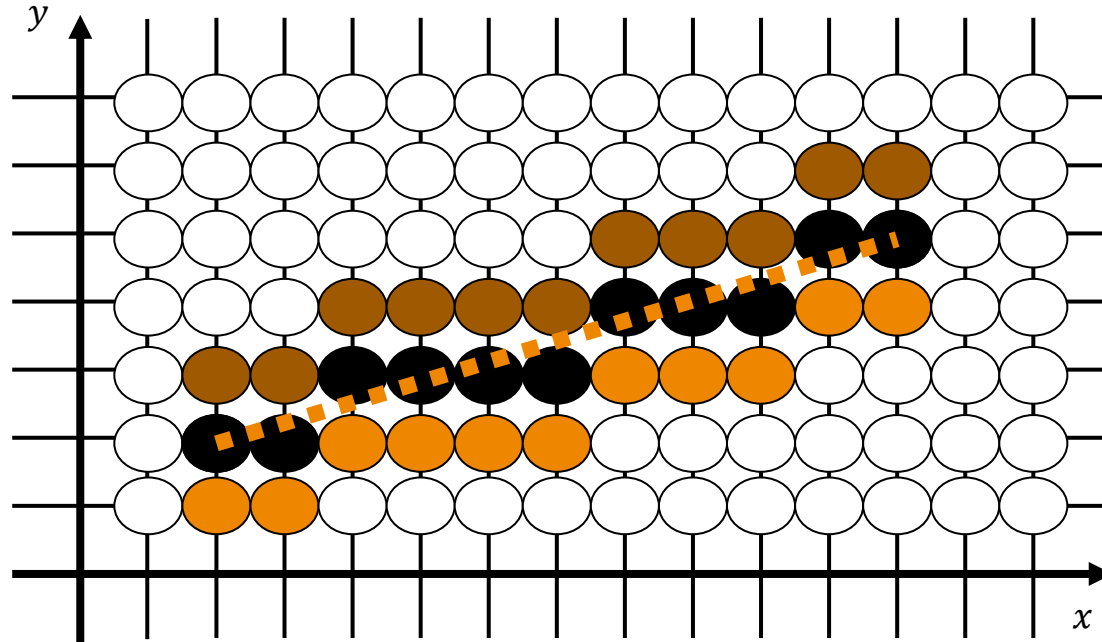
Pixelwiederholungen – Breite=2

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut:



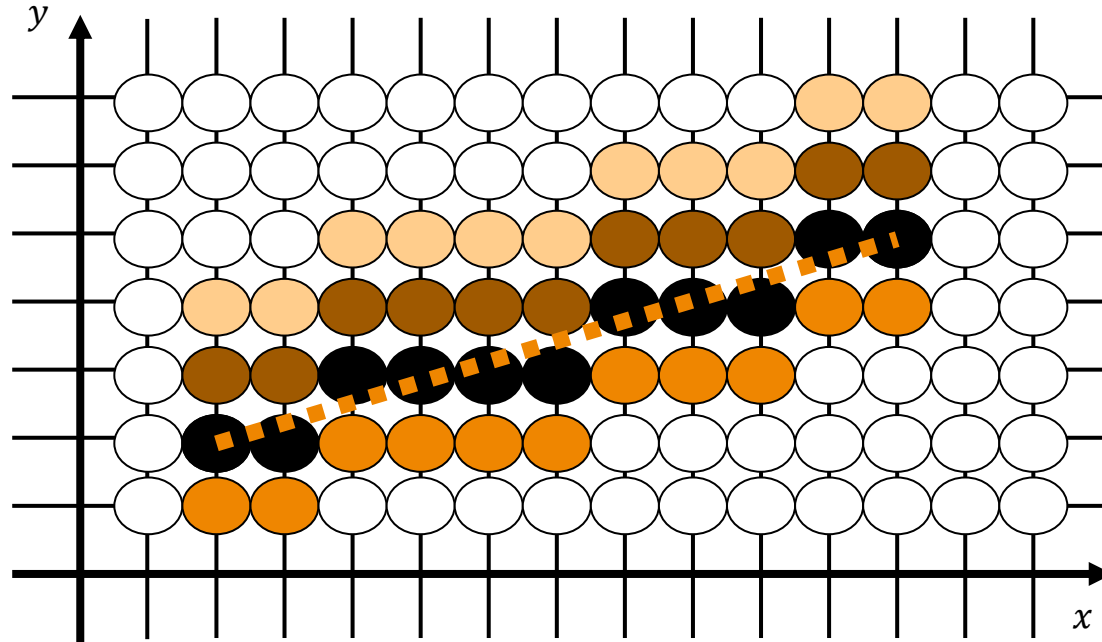
Pixelwiederholungen – Breite=3

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut:



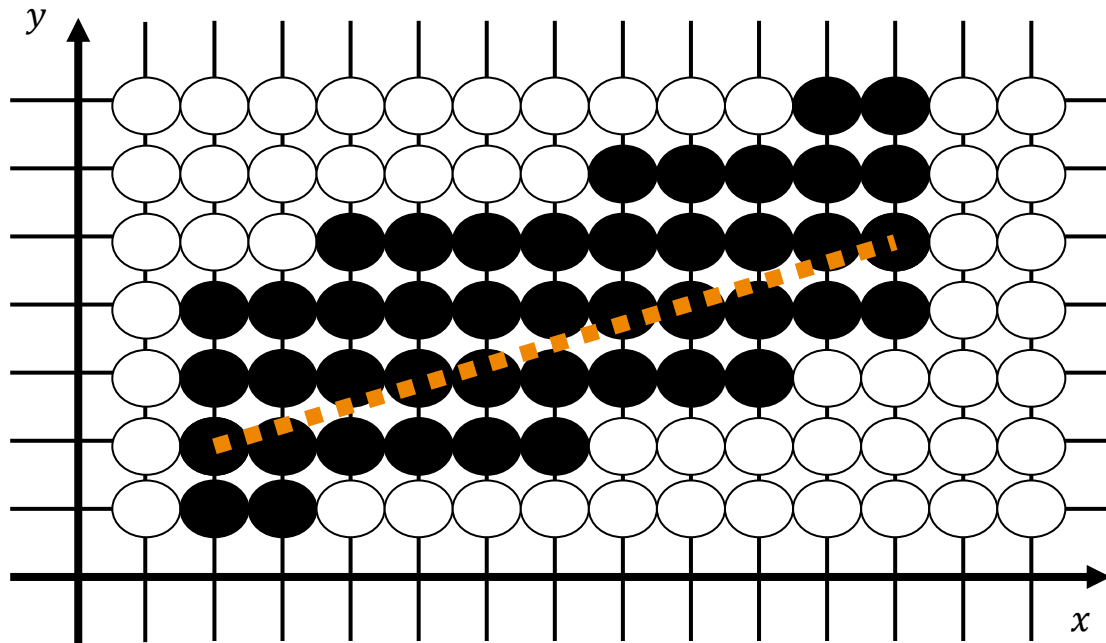
Pixelwiederholungen – Breite=4

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut:



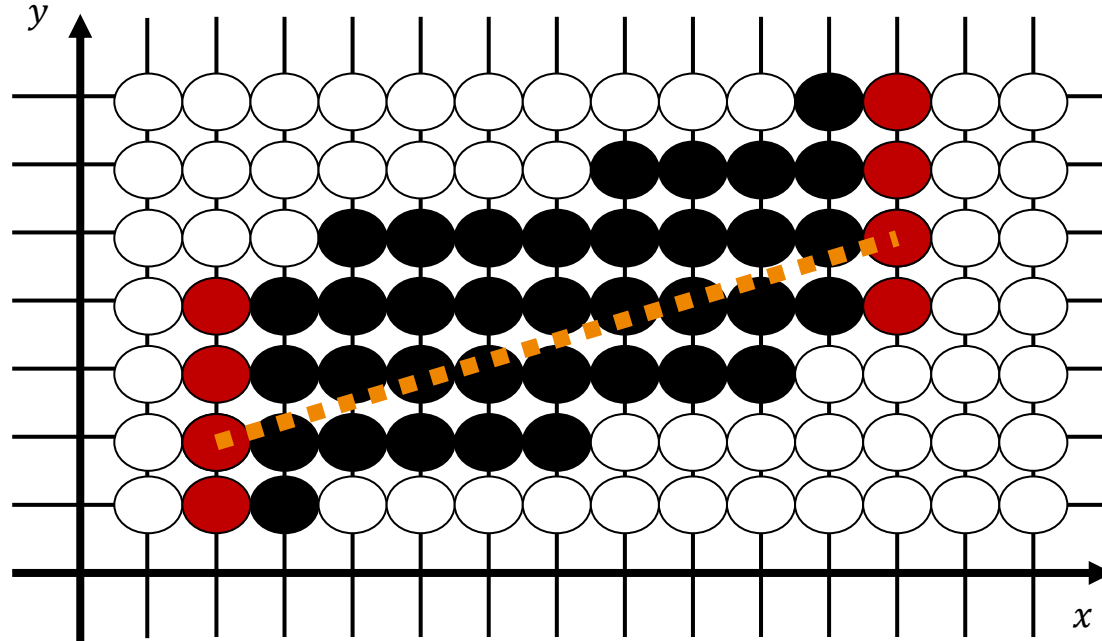
Achsenorientierte Pixel-Duplikation

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut:



Achsenorientierte Pixel-Duplikation – Problematik

Das Duplizieren von Pixeln (achsenorientierte Pixel-Duplikation) funktioniert für Linien oft ganz gut:



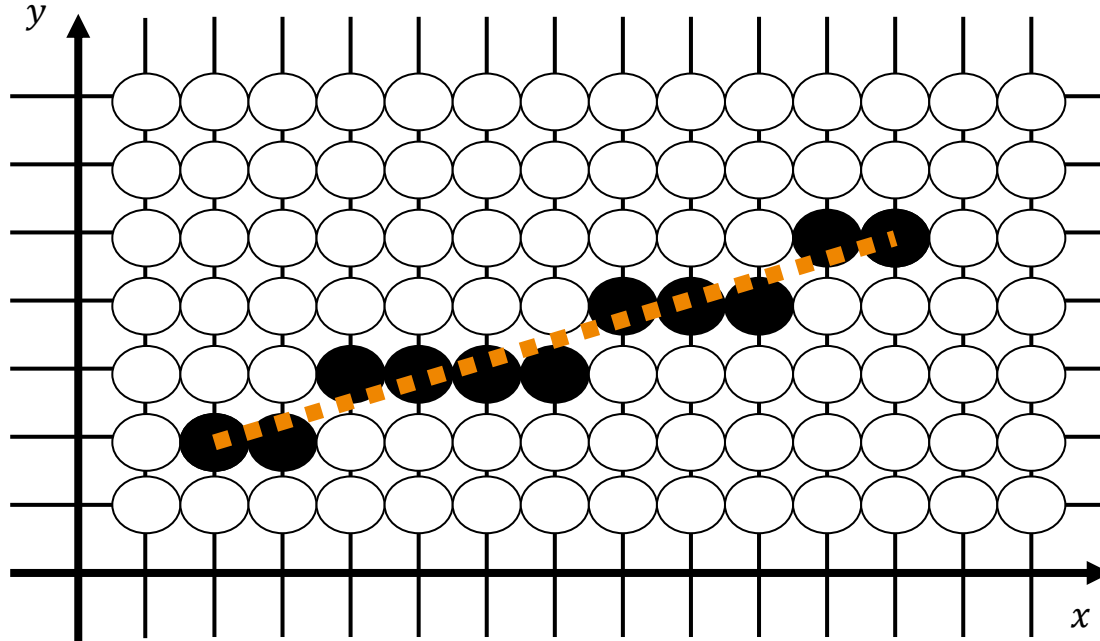
Linienenden sind immer horizontal/vertikal. Schräge Linien wirken damit kürzer.



Wie könnten wir es geschickter lösen?

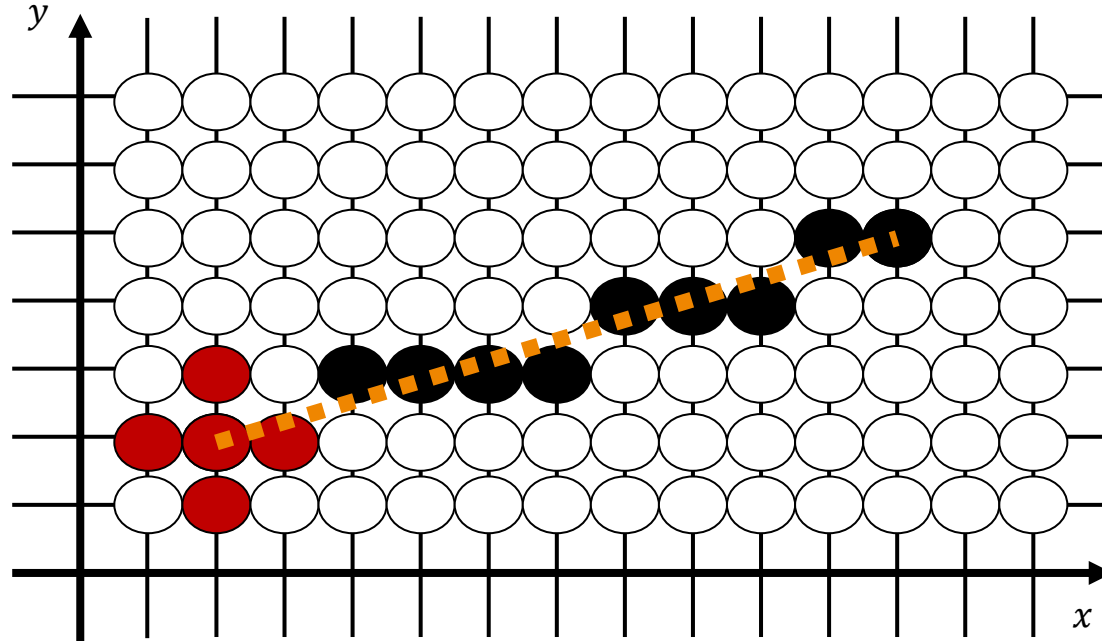
Zuerst Bresenham-Mittellinie zeichnen

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



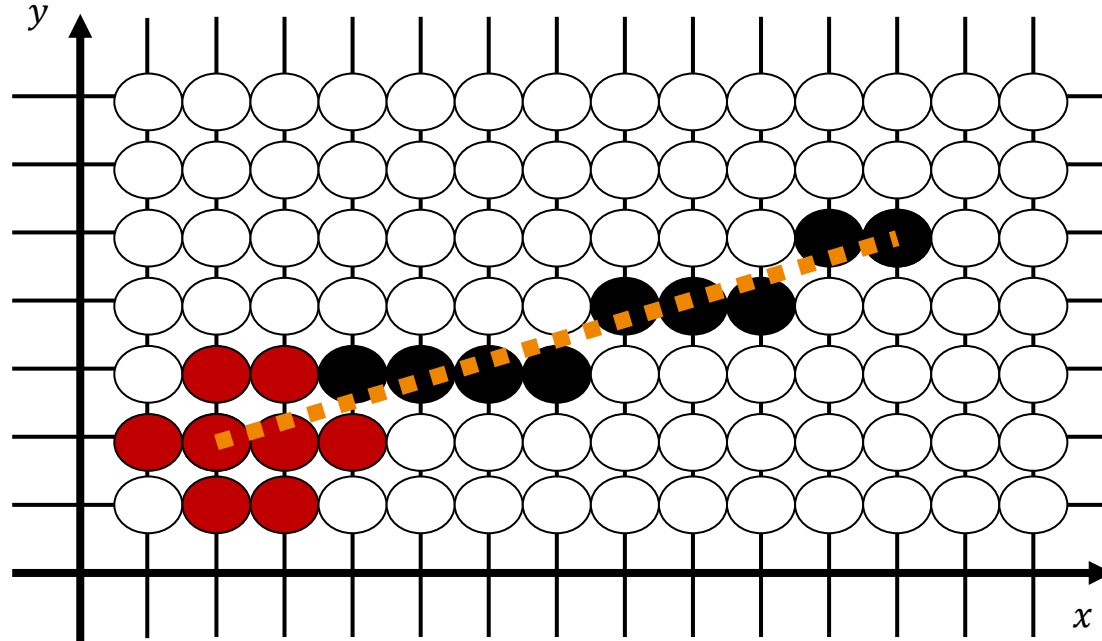
Um jeden Linienpunkt wird ein Kreis gezeichnet

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



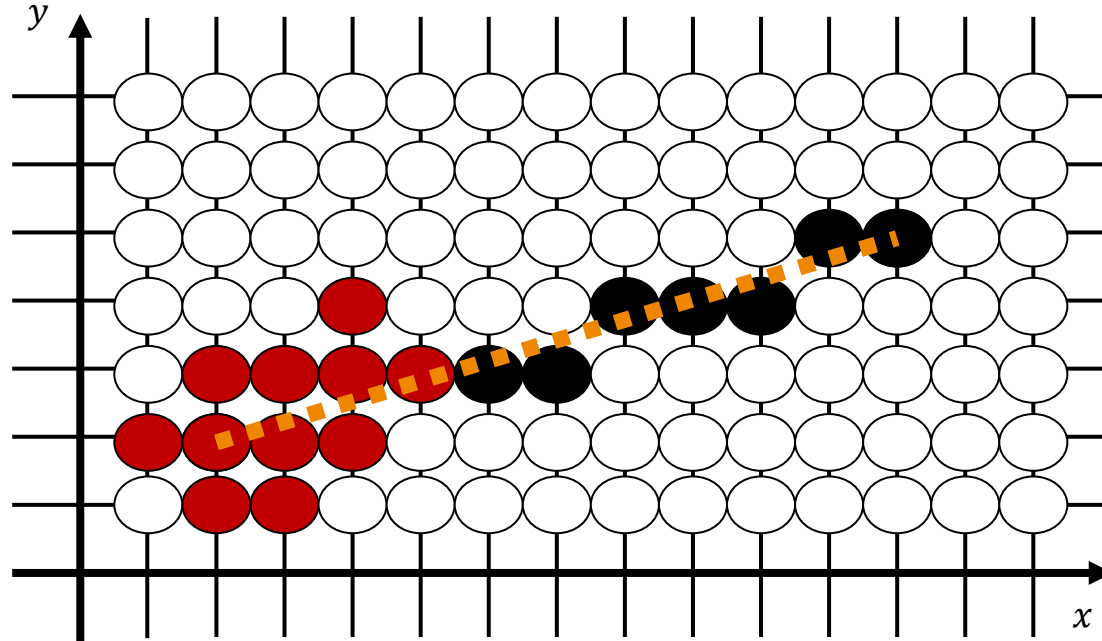
Um jeden Linienpunkt wird ein Kreis gezeichnet

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



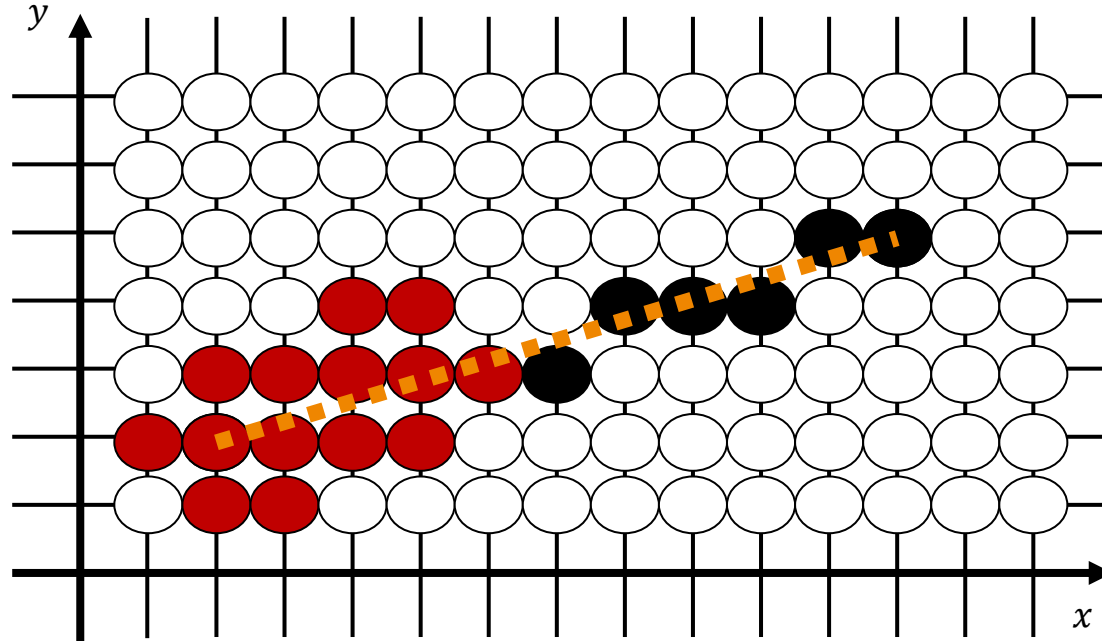
Um jeden Linienpunkt wird ein Kreis gezeichnet

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



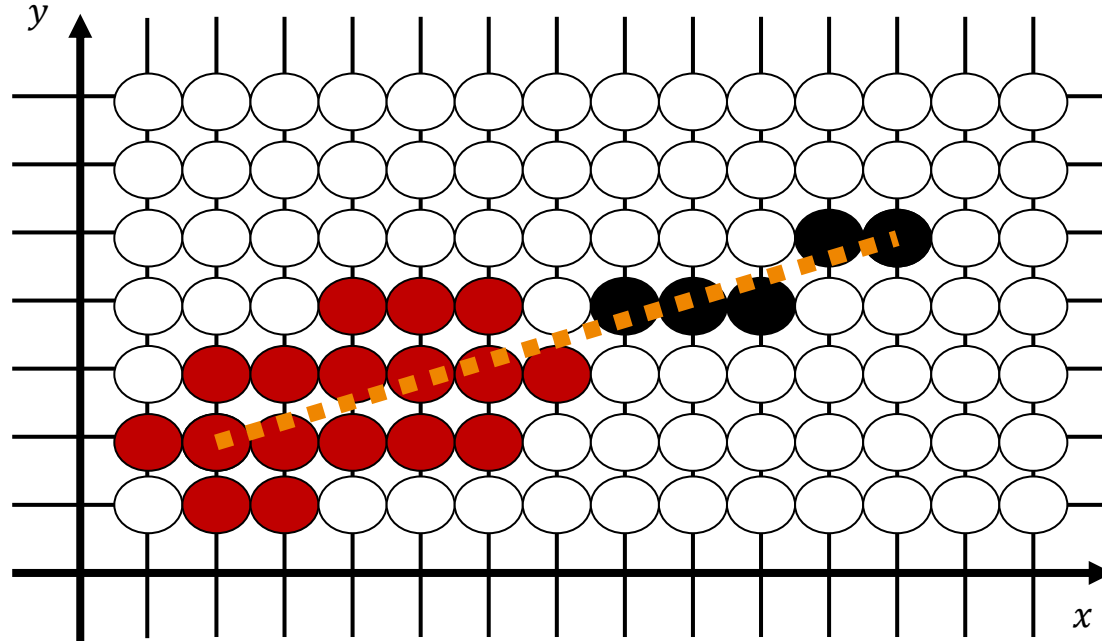
Um jeden Linienpunkt wird ein Kreis gezeichnet

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



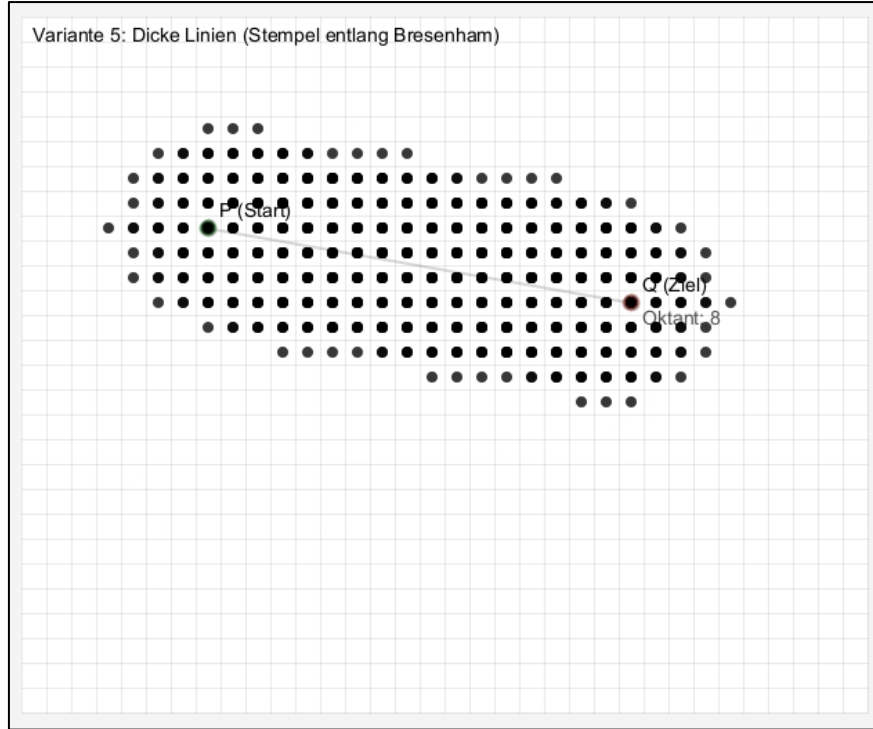
Um jeden Linienpunkt wird ein Kreis gezeichnet

Eine bessere Möglichkeit lässt sich über den folgenden Algorithmus realisieren:



Unterschiedlich dicke Linien zeichnen

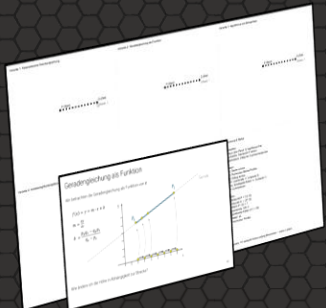
Schauen wir uns das hier auch noch einmal in einem Beispiel an.



Linien unter der Lupe

Key Takeaways

- Rasterdarstellung von Linien ist nicht eindeutig
- Raster ist eine Annäherung
- Parametrisierte Geradengleichung als Basis: Streckengleichung, Schrittweite, viel Gleitkommaarithmetik
- Geradengleichung als Funktion: Halbierung der Gleitkommaarithmetik, Achsentausch bei steilen Linien
- Bresenham arbeitet mit Ganzzahlen: effizient, Entscheidung lokal
- Anti-Aliasing: Überlappungskoeffizienten



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Wie muss die parametrisierte Geradengleichung angepasst werden, um daraus eine parametrisierte Streckengleichung zu formulieren?
- (2) Leiten Sie die Ergebnisse der Parameter m und b für $f(x) = mx + b$ Schritt für Schritt her. Verwenden Sie dafür das abstrakte Beispiel aus der Vorlesung mit den Punkten P und Q . Berechnen Sie weiterhin m und b jeweils für die konkreten Punktepaaire $\{P_1(2,3), Q_1(4,6)\}$, $\{P_2(1,1), Q_2(2,1)\}$ und $\{P_3(2.3,3.1), Q_3(4.2,3.9)\}$.
- (3) Erläutern Sie die schrittweise Verbesserung der fünf vorgestellten Varianten zum Linienzeichnen und benennen Sie diese.
- (4) Angenommen, es gibt einen Punkt $P = (x_1, y_1)$ der auf der Linie liegt. Überlege Sie sich, ob sich hier auch $b = y_1 - mx_1$ verwenden ließe.
- (5) Wenden Sie den Bresenham-Algorithmus (analog zum Vorlesungsbeispiel) schrittweise zum Zeichnen der Linie $P_2(12,2), Q_2(2,7)$ an und notieren Sie sich dabei die Entwicklung des Fehlers.
- (6) Erläutern Sie den Begriff Aliasing und in diesem Zusammenhang auch den Begriff Antialiasing.
- (7) Wenden Sie den Bresenham-Algorithmus schrittweise zum Zeichnen der Linie von $P(5,1)$ nach $Q(1,8)$ an und notieren Sie sich dabei die Entwicklung des Fehlers.
- (8) Verwenden Sie die Ergebnisse aus Aufgabe 7 und leiten sie die Überlappungskoeffizienten ab. Liefern sie eine Lösung (Skizze) für die Linie aus Aufgabe 7 mit Antialiasing.

1. t auf $[0,1]$ beschränken (Annahme: Geradengleichung ist $P + t \cdot \vec{v}$ mit $\vec{v} = Q - P$ * Strecke $\overrightarrow{PQ}$)

2. $P = (p_x, p_y)$, $Q = (q_x, q_y)$

$$y = mx + b$$

$$I: p_y = m p_x + b$$

$$II: q_y = m q_x + b$$

$$III: q_y - p_y = m q_x - m p_x = m(q_x - p_x) \quad | : (q_x - p_x)$$

$$m = \frac{q_y - p_y}{q_x - p_x}$$

$$m \text{ in } I: p_y = \frac{q_y - p_y}{q_x - p_x} p_x + b \quad | - \frac{q_y - p_y}{q_x - p_x} p_x$$

$$b = p_y - \frac{q_y - p_y}{q_x - p_x} p_x = \frac{q_x \cdot p_y - p_x \cdot q_y}{q_x - p_x} = \frac{q_x \cdot p_y - p_x \cdot q_y}{q_x - p_x} = \frac{q_x \cdot p_y - p_x \cdot q_y}{q_x - p_x}$$

$\{P_1(2,3), Q_1(4,6)\}$, $\{P_2(1,1), Q_2(2,1)\}$ und $\{P_3(2.3,3.1), Q_3(4.2,3.9)\}$.

$$m_1 = \frac{6-3}{4-2} = \frac{3}{2}$$

$$m_2 = \frac{1-1}{2-1} = 0$$

$$m_3 = \frac{3.9-3.1}{4.2-2.3} = \frac{0.8}{1.9} = \frac{8}{19}$$

$$\left(\frac{3.1-4.9}{4.0-4.9} = \frac{42.0-3.1}{43.0} = \frac{5.9}{43.0} \right)$$

$$b_1 = \frac{6-3}{4-2} = \frac{12-12}{2} = 0$$

$$b_2 = p_y - m p_x = p_{y2} = 1$$

$$b_3 = p_y - m p_x = 3.1 - \frac{8}{19} \cdot 2.3 = 3.1 - \frac{8}{19} \cdot \frac{23}{10} = \frac{31}{10} - \frac{184}{190} = \frac{589}{190} - \frac{184}{190} = \frac{405}{190} = \frac{81}{38}$$

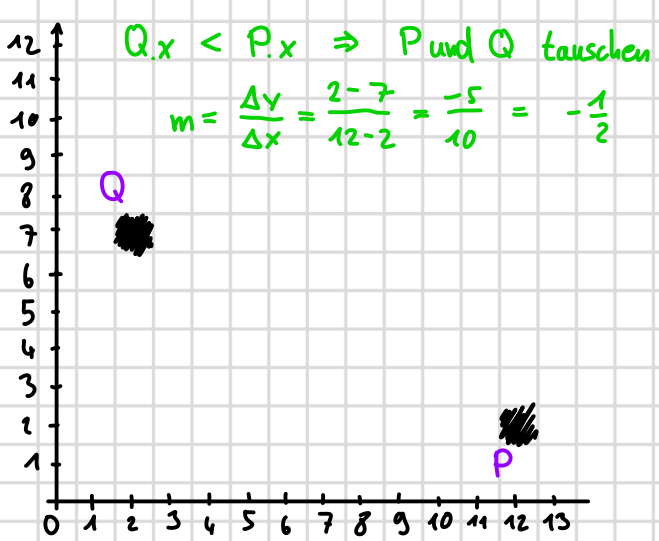
3. - Param. Geradengl. $\rightarrow$ Funktion: reduziere Float-Ops um $\frac{1}{2}$

- gleichbleibende Berechnungen vor die Schleife

- Funktion $\rightarrow$ Bresenham:

4. $P = (x_1, y_1)$ geht auch $b = y_1 - m x_1$?

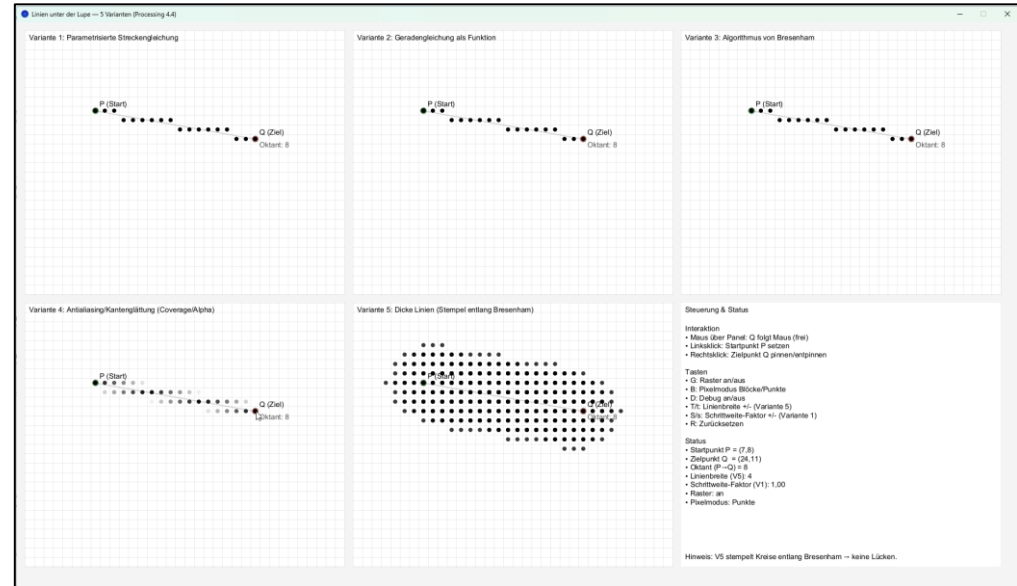
denke schon



Praktische Aufgaben

Die praktischen Aufgaben dienen zur Vertiefung des Stoffs und sollten selbstständig bearbeitet werden.

Aufgabe 1) [Processing] Überlegen Sie sich ein geeignetes Vorgehen, um die Performance zwischen den fünf vorgestellten Varianten in der Praxis zu testen und führen Sie Zeitmessungen durch.





Vielen Dank für die Aufmerksamkeit